ỦY BAN NHÂN DÂN TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP

MÔN NGÔN NGỮ LẬP TRÌNH C#

LAB05: LẬP TRÌNH WINFORM NÂNG CAO

Giảng viên hướng dẫn: **ĐỖ NHƯ TÀI**

Sinh viên thực hiện: **BÙI PHÚC HƯNG**

Mã số sinh viên: **3124411115**

TP. HỒ CHÍ MINH, THÁNG 10 NĂM 2026

THỰC HÀNH 5C: LẬP TRÌNH WINFORM NÂNG CAO {1}

A. MỤC TIÊU:

− Thiết kế được giao diện Windows Form từ các control trong C#.

− Sử dụng được các Controls Textbox, Label, Button, Listbox, Combobox, TreeView, TabControl

B. NỘI DUNG THỰC HÀNH

Kiến thức cần nhớ:

− Listbox:

+ Thuộc tính: Items, SelectionMode, SelectedIndex, SelectedItem.

+ Sự kiện: SelectedIndexChanged.

+ Phoɹơng thức: Add, AddRange, RemoveAt, Remove, IndexOf.

− ComboBox:

+ Thuộc tính: Items, DropDownStyle, SelectedIndex, SelectedItems, Text.

+ Sự kiện: SelectedIndexChanged.

− TreeView:

+ Thuộc tính: Nodes, SelectedNode, ShowRootLine, ImageList, ShowLine, ImageIndex, SelectedImageIndex, FirstNode, LastNode, NextNode, PrevNode, Text.

+ Sự kiện: AfterSelect, Click

+ Phương thức: CollapseAll, ExpandAll.

Bài 01:

1. Yêu cầu đề

Form: Text = "Combobox", KeyPreview = true

TextBox: txtSo (Nhập số)

ComboBox: cboSo

ListBox: lstUocSo (Danh sách các ước số)

Buttons:

- btnCapNhat: Text = "Cập nhật"
- btnTong: Text = "Tổng các ước số”

- btnUocSoChan: Text = "Số lượng các ước số chẵn"
- btnUocSoNT: Text = "Số lượng các ước số nguyên tố"
- btnThoat: Text = "Thoát"

2. Mã nguồn

Bai1.cs

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace Bai1
{
    public class UocSo
    {
        // Kiểm tra một số nguyên có phải là số nguyên tố ko
        public static bool LaSoNguyenTo(int n)
        {
            if (n < 2) return false;
            for (int i = 2; i <= Math.Sqrt(n); i++)
            {
                if (n % i == 0) return false;
            }
            return true;
        }

        // Lấy danh sách các ước số nguyên dương của n
        public static List<int> LayDanhSachUocSo(int n)
        {
            List<int> danhSach = new List<int>();
            n = Math.Abs(n);
            if (n == 0) return danhSach;

            for (int i = 1; i <= n; i++)
            {
                if (n % i == 0)
                {
                    danhSach.Add(i);
                }
            }
            return danhSach;
        }

        // Tính tổng các ước số
        public static int TinhTong(List<int> danhSach) => danhSach.Sum();

        // Đếm số lượng ước số chẵn
        public static int DemSoChan(List<int> danhSach) => danhSach.Count(x
=> x % 2 == 0);
```

```
        // Đếm số lượng ước số nguyên tố
        public static int DemSoNguyenTo(List<int> danhSach) =>
danhSach.Count(LaSoNguyenTo);
    }
}
```

GiaoDienBai1.cs

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Linq;
using System.Windows.Forms;

namespace Bai1
{
    public class GiaoDienBai1 : Form
    {
        private Label lblNhapSo, lblDanhSachUoc;
        private TextBox txtNhapSo;
        private ComboBox cboSo;
        private ListBox lstUocSo;
        private Button btnCapNhat, btnTongUoc, btnSoLuongChan,
btnSoLuongNguyenTo, btnThoat;

        public GiaoDienBai1()
        {
            KhoiTaoGiaoDien();
        }

        private void KhoiTaoGiaoDien()
        {
            this.Text = "Combobox";
            this.Size = new Size(520, 300);
            this.StartPosition = FormStartPosition.CenterScreen;
            this.FormBorderStyle = FormBorderStyle.FixedDialog;
            this.MaximizeBox = false;

            // 1. Bên trái: Nhập số và ComboBox
            lblNhapSo = new Label() { Text = "Nhập Số", Location = new
Point(70, 20), AutoSize = true, Font = new Font("Arial", 11,
FontStyle.Regular) };

            txtNhapSo = new TextBox() { Location = new Point(70, 48), Size =
new Size(95, 23) };
            txtNhapSo.KeyPress += TxtNhapSo_KeyPress;
            txtNhapSo.KeyDown += TxtNhapSo_KeyDown;

            // Nút Cập nhật với HotKey Alt + C
```

```
            btnCapNhat = new Button() { Text = "&Cập nhật", Location = new
Point(175, 45), Size = new Size(75, 28) };
            btnCapNhat.Click += BtnCapNhat_Click;

            cboSo = new ComboBox()
            {
                Location = new Point(70, 95),
                Size = new Size(180, 23),
                DropDownStyle = ComboBoxStyle.DropDownList
            };
            cboSo.SelectedIndexChanged += CboSo_SelectedIndexChanged;

            // Nút Thoát với HotKey Alt + T
            btnThoat = new Button() { Text = "&Thoát", Location = new
Point(175, 215), Size = new Size(75, 30) };
            btnThoat.Click += (s, e) => this.Close();

            // 2. Bên phải: Danh sách ước số
            lblDanhSachUoc = new Label() { Text = "Danh sách các ước số",
Location = new Point(285, 20), AutoSize = true, Font = new Font("Arial", 11,
FontStyle.Regular) };

            lstUocSo = new ListBox() { Location = new Point(285, 48), Size =
new Size(185, 85) };

            // 3. Các nút thống kê bên dưới ListBox với HotKey tương ứng
            // Alt + T cho Tổng các ước số
            btnTongUoc = new Button() { Text = "&Tổng các ước số", Location
= new Point(285, 140), Size = new Size(185, 28) };
            // Alt + S cho Số lượng các ước số chẵn
            btnSoLuongChan = new Button() { Text = "&Số lượng các ước số
chẵn", Location = new Point(285, 175), Size = new Size(185, 28) };
            // Alt + N cho Số lượng các ước số nguyên tố
            btnSoLuongNguyenTo = new Button() { Text = "Số lượng các ước số
&nguyên tố", Location = new Point(285, 210), Size = new Size(185, 28) };

            btnTongUoc.Click += BtnTongUoc_Click;
            btnSoLuongChan.Click += BtnSoLuongChan_Click;
            btnSoLuongNguyenTo.Click += BtnSoLuongNguyenTo_Click;

            this.Controls.AddRange(new Control[] {
                lblNhapSo, txtNhapSo, btnCapNhat, cboSo, btnThoat,
                lblDanhSachUoc, lstUocSo, btnTongUoc, btnSoLuongChan,
btnSoLuongNguyenTo
            });

            this.FormClosing += GiaoDienBai1_FormClosing;
        }

        // Chặn nhập chữ cái, chỉ nhận số nguyên dương
        private void TxtNhapSo_KeyPress(object sender, KeyPressEventArgs e)
```

```
        {
            if (!char.IsControl(e.KeyChar) && !char.IsDigit(e.KeyChar))
            {
                e.Handled = true;
            }
        }

        // Nhấn phím Enter để cập nhật số nhanh
        private void TxtNhapSo_KeyDown(object sender, KeyEventArgs e)
        {
            if (e.KeyCode == Keys.Enter)
            {
                BtnCapNhat_Click(sender, e);
                e.SuppressKeyPress = true;
            }
        }

        // Nút Cập nhật: Thêm số vào ComboBox (kiểm tra rỗng, kiểm tra
trùng)
        private void BtnCapNhat_Click(object sender, EventArgs e)
        {
            if (string.IsNullOrWhiteSpace(txtNhapSo.Text))
            {
                MessageBox.Show("Vui lòng nhập một số nguyên dương!", "Thông
báo", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                txtNhapSo.Focus();
                return;
            }

            int so = int.Parse(txtNhapSo.Text.Trim());
            if (so <= 0)
            {
                MessageBox.Show("Vui lòng nhập số nguyên lớn hơn 0!", "Lỗi
giá trị", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                txtNhapSo.Focus();
                return;
            }

            // Kiểm tra số vừa nhập đã tồn tại trong Combobox chưa
            if (cboSo.Items.Contains(so))
            {
                MessageBox.Show("Số này đã tồn tại trong danh sách!", "Trùng
dữ liệu", MessageBoxButtons.OK, MessageBoxIcon.Information);
            }
            else
            {
                cboSo.Items.Add(so);
                cboSo.SelectedItem = so; // Tự động chọn số vừa thêm
            }

            txtNhapSo.Clear();
```

```
            txtNhapSo.Focus();
        }

        // Khi người dùng chọn một số trong ComboBox: Liệt kê ước số vào
ListBox
        private void CboSo_SelectedIndexChanged(object sender, EventArgs e)
        {
            if (cboSo.SelectedItem != null)
            {
                int soDuocChon = (int)cboSo.SelectedItem;
                List<int> danhSachUoc = UocSo.LayDanhSachUocSo(soDuocChon);

                lstUocSo.Items.Clear();
                foreach (int uoc in danhSachUoc)
                {
                    lstUocSo.Items.Add(uoc);
                }
            }
        }

        // Lấy danh sách ước hiện tại từ ListBox
        private List<int> LayDanhSachHienTai()
        {
            List<int> ds = new List<int>();
            foreach (var item in lstUocSo.Items)
            {
                ds.Add((int)item);
            }
            return ds;
        }

        // Nút Tổng các ước số
        private void BtnTongUoc_Click(object sender, EventArgs e)
        {
            if (lstUocSo.Items.Count == 0)
            {
                MessageBox.Show("Chưa có ước số nào để tính toán!", "Thông
báo", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                return;
            }
            int tong = UocSo.TinhTong(LayDanhSachHienTai());
            MessageBox.Show($"Tổng các ước số là: {tong}", "Kết quả",
MessageBoxButtons.OK, MessageBoxIcon.Information);
        }

        // Nút Số lượng các ước số chẵn
        private void BtnSoLuongChan_Click(object sender, EventArgs e)
        {
            if (lstUocSo.Items.Count == 0)
            {
```

```
                MessageBox.Show("Chưa có ước số nào để tính toán!", "Thông
báo", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                return;
            }
            int demChan = UocSo.DemSoChan(LayDanhSachHienTai());
            MessageBox.Show($"Số lượng ước số chẵn là: {demChan}", "Kết
quả", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }

        // Nút Số lượng các ước số nguyên tố
        private void BtnSoLuongNguyenTo_Click(object sender, EventArgs e)
        {
            if (lstUocSo.Items.Count == 0)
            {
                MessageBox.Show("Chưa có ước số nào để tính toán!", "Thông
báo", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                return;
            }
            int demNT = UocSo.DemSoNguyenTo(LayDanhSachHienTai());
            MessageBox.Show($"Số lượng ước số nguyên tố là: {demNT}", "Kết
quả", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }

        // Xác nhận khi thoát chuong trình
        private void GiaoDienBai1_FormClosing(object sender,
FormClosingEventArgs e)
        {
            DialogResult hoi = MessageBox.Show("Bạn có muốn thoát chương
trình không?", "Xác nhận", MessageBoxButtons.YesNo,
MessageBoxIcon.Question);
            if (hoi == DialogResult.No)
            {
                e.Cancel = true;
            }
        }
    }
}
```

Program.cs

```
using System;
using System.Windows.Forms;

namespace Bai1
{
    static class Program
    {
        static void Main()
        {
            Application.EnableVisualStyles();
```

```
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new GiaoDienBai1());
        }
    }
}
```

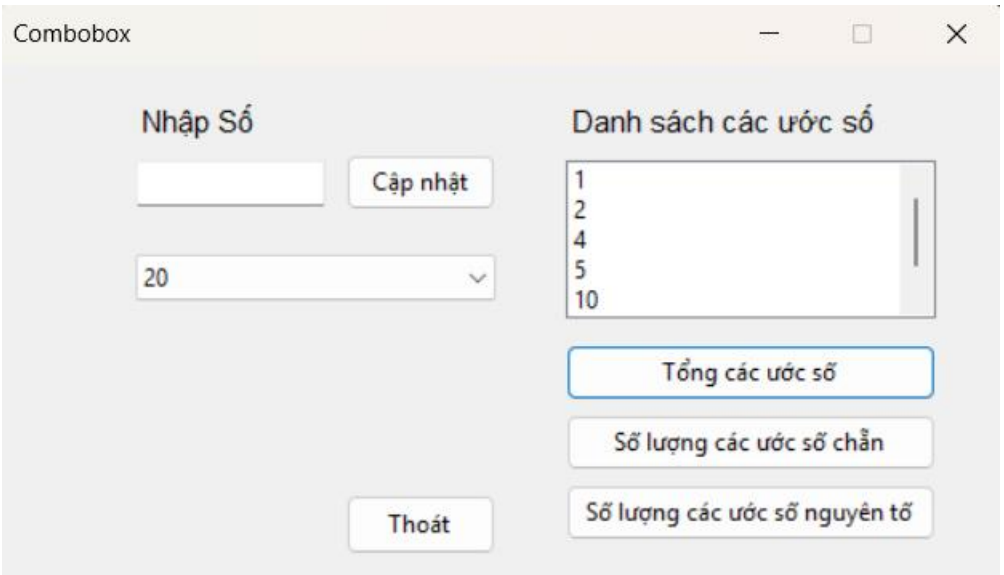
3. Giải thích bài làm

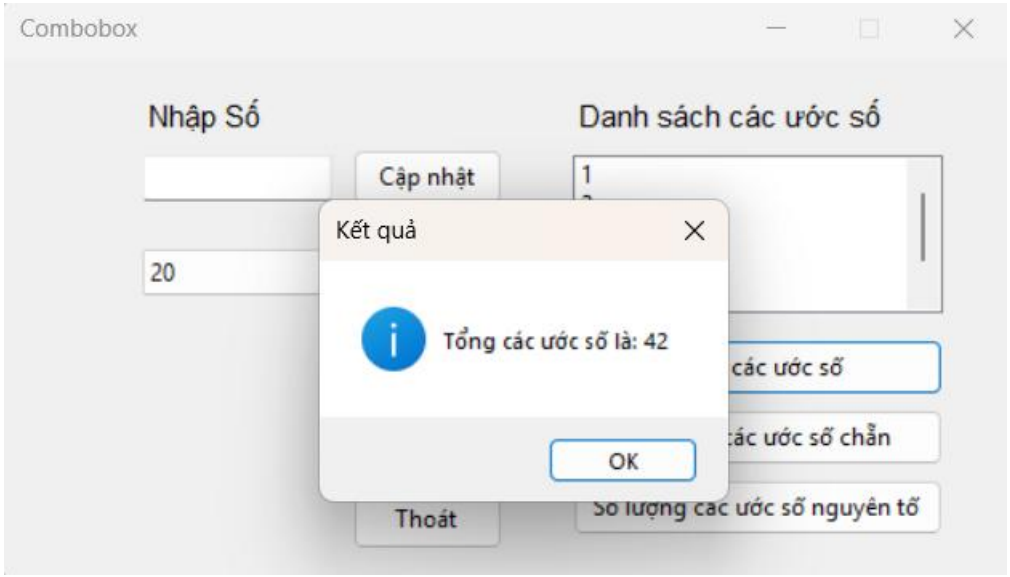
Lớp Bai1: Đóng gói hàm LayDanhSachUocSo(n) duyệt từ 1 → n lấy các số chia hết. Kết hợp các phương thức LINQ (Sum, Count) và hàm kiểm tra số nguyên tố LaSoNguyenTo() để thực hiện nhanh các tác vụ thống kê.

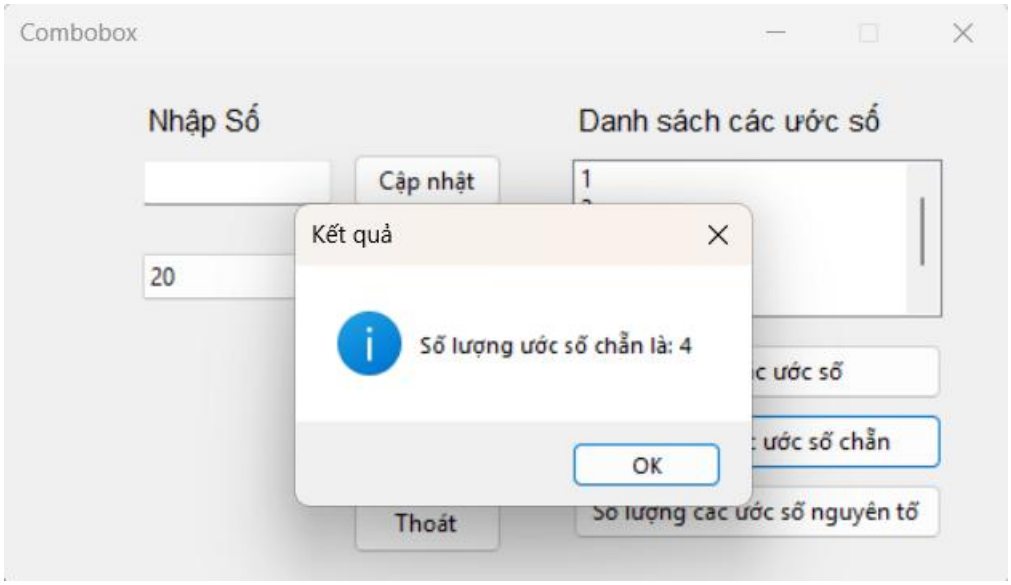
Liên kết dữ liệu giữa ComboBox và ListBox: Sự kiện SelectedIndexChanged của cboSo được kích hoạt ngay khi người dùng bấm chọn số. Danh sách ước số được lấy ra và đổ vào lstUocSo.Items thông qua vòng lặp.

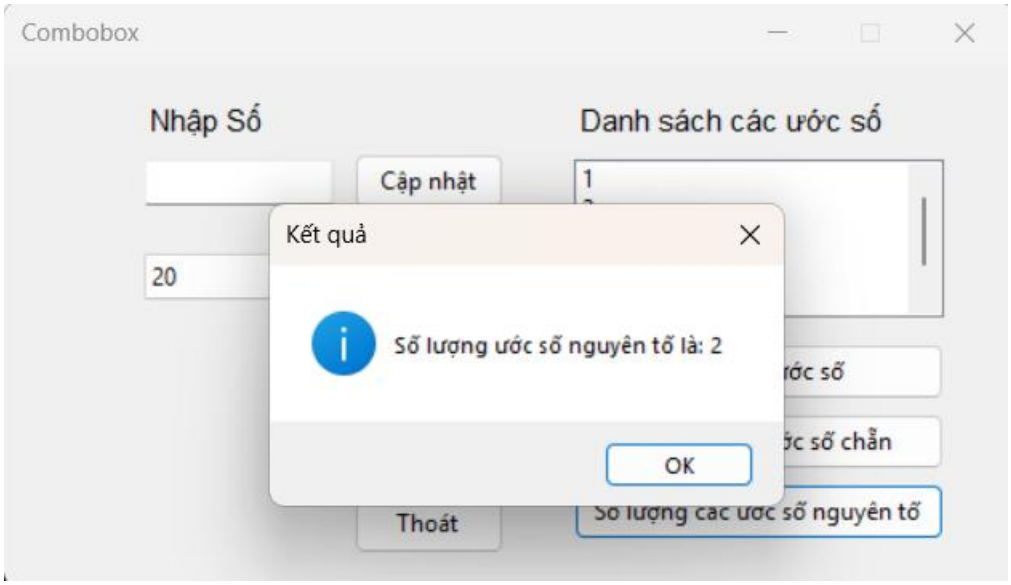
Kiểm tra trùng và phím tắt: Nút **Cập nhật** dùng cboSo.Items.Contains() để chống thêm số trùng. Thuộc tính Text của các nút bấm được chèn dấu & (ví dụ: &Cập nhật, &Thoát, &Tổng các ước số, &Số lượng các ước số chẵn, Số lượng các ước số &nguyên tố) để người dùng có thể kích hoạt nhanh bằng tổ hợp Alt + ký tự

4. Kết quả




Combobox
Nhập Số
Danh sách các ước số
Cập nhật
20
1
Kết quả
Tổng các ước số là: 42
OK
các ước số
các ước số chẵn
Thoát
Số lượng các ước số nguyên tố


Combobox
Nhập Số
Danh sách các ước số
Cập nhật
20
1
Kết quả
Số lượng ước số chẵn là: 4
OK
ước số
ước số chẵn
Thoát
Số lượng các ước số nguyên tố


Combobox
Nhập Số
Danh sách các ước số
Cập nhật
20
1
Kết quả
Số lượng ước số nguyên tố là: 2
OK
Thoát
Số lượng các ước số nguyên tố

Bài 02:

1. **Yêu cầu đề**

− Khi form khởi động, nạp sẵn danh sách một số lớp mẫu (05DHTH1, 05DHTH2, 05DHTH3, 05DHTH4) vào cả TreeView và ComboBox.

− Ô CheckBox Thêm lớp mặc định là false thì khung Thông tin lớp được ẩn đi; khi người dùng tích chọn thì khung này hiện lên.

− Nút Thêm lớp: Thêm một lớp mới vào TreeView và ComboBox, có kiểm tra trùng tên lớp trước khi thêm.

− Nút Cập Nhật: Thêm sinh viên vào lớp đang chọn trên ComboBox với định dạng nút cha là Mã SV, Họ Tên và nút con bên dưới là Địa Chỉ. Kiểm tra không được để trống các ô nhập và không được trùng mã sinh viên.

− Khi click chọn một sinh viên trên TreeView, tự động hiển thị thông tin chi tiết của sinh viên đó lên các ô TextBox tương ứng.

− Nút Xóa: Chỉ cho phép xóa đúng nút sinh viên đang chọn trên TreeView, có hiển thị hộp thoại xác nhận trước khi xóa.

− Hỏi xác nhận trước khi đóng Form

2. **Mã nguồn**

Bai2.cs

```
using System;
using System.Collections.Generic;

namespace ThucHanh5c_Bai2
{
    // Lớp chứa thông tin một sinh viên
    public class SinhVien
    {
        public string MaSV { get; set; }
        public string HoTen { get; set; }
        public string DiaChi { get; set; }

        public SinhVien()
        {
            MaSV = "";
            HoTen = "";
            DiaChi = "";
        }

        public SinhVien(string maSV, string hoTen, string diaChi)
```

```
        {
            MaSV = maSV;
            HoTen = hoTen;
            DiaChi = diaChi;
        }

        // Định dạng chuỗi hiển thị trên nút cha: "Mã SV, Họ Tên"
        public string LayChuoiHienThi()
        {
            return MaSV + ", " + HoTen;
        }
    }

    // Lớp quản lý danh sách lớp và sinh viên
    public class QuanLyLopHoc
    {
        // Danh sách tên các lớp học
        private List<string> danhSachLop;

        // Lưu trữ sinh viên: Khóa là tên lớp, Giá trị là danh sách sinh
viên của lớp đó
        private Dictionary<string, List<SinhVien>> duLieuSinhVien;

        public QuanLyLopHoc()
        {
            danhSachLop = new List<string>();
            duLieuSinhVien = new Dictionary<string, List<SinhVien>>();

            // Nạp sẵn 4 lớp mẫu ban đầu theo đề bài
            string[] cacLop = { "05DHTH1", "05DHTH2", "05DHTH3", "05DHTH4"
};
            foreach (string lop in cacLop)
            {
                ThemLop(lop);
            }
        }

        // Lấy danh sách các lớp hiện có
        public List<string> LayDanhSachLop()
        {
            return danhSachLop;
        }

        // Kiểm tra xem tên lớp đã tồn tại chưa (không phân biệt chữ hoa,
chữ thường)
        public bool KiemTraTrungLop(string tenLop)
        {
            foreach (string lop in danhSachLop)
            {
                if (string.Compare(lop, tenLop, true) == 0)
                {
```

```
                return true;
            }
        }
        return false;
    }

    // Thêm lớp mới
    public bool ThemLop(string tenLop)
    {
        if (KiemTraTrungLop(tenLop)) return false;

        danhSachLop.Add(tenLop);
        duLieuSinhVien[tenLop] = new List<SinhVien>();
        return true;
    }

    // Kiểm tra xem mã sinh viên đã có trong toàn trường chưa
    public bool KiemTraTrungMaSV(string maSV)
    {
        foreach (var dsSV in duLieuSinhVien.Values)
        {
            foreach (var sv in dsSV)
            {
                if (string.Compare(sv.MaSV, maSV, true) == 0)
                {
                    return true;
                }
            }
        }
        return false;
    }

    // Thêm sinh viên vào lớp chỉ định
    public bool ThemSinhVien(string tenLop, SinhVien sv)
    {
        if (!duLieuSinhVien.ContainsKey(tenLop)) return false;
        if (KiemTraTrungMaSV(sv.MaSV)) return false;

        duLieuSinhVien[tenLop].Add(sv);
        return true;
    }

    // Xóa một sinh viên theo mã SV
    public bool XoaSinhVien(string maSV)
    {
        foreach (var dsSV in duLieuSinhVien.Values)
        {
            for (int i = 0; i < dsSV.Count; i++)
            {
                if (string.Compare(dsSV[i].MaSV, maSV, true) == 0)
                {
```

```
                        dsSV.RemoveAt(i);
                        return true;
                    }
                }
            }
            return false;
        }
    }
}
```

GiaoDienBai2.cs

```
using System;
using System.Drawing;
using System.Windows.Forms;

namespace Bai2
{
    public class GiaoDienBai2 : Form
    {
        // Khai báo các điều khiển trên Form theo đề bài
        private TreeView trvLop;
        private ComboBox cboLop;
        private Label lblChonLop;

        // Nhóm thông tin sinh viên
        private GroupBox grpTTSV;
        private Label lblMaSV, lblHoTen, lblDiaChi;
        private TextBox txtMaSV, txtHoTen, txtDiaChi;
        private Button btnCapNhat, btnXoa;

        // Nhóm thêm lớp mới
        private CheckBox chkThemLop;
        private GroupBox grpThemLop;
        private Label lblTenLop;
        private TextBox txtTenLop;
        private Button btnThemLop;

        public GiaoDienBai2()
        {
            KhoiTaoGiaoDien();
        }

        private void KhoiTaoGiaoDien()
        {
            this.Text = "Quản lý sinh viên";
            this.Size = new Size(680, 500);
            this.StartPosition = FormStartPosition.CenterScreen;
            this.FormBorderStyle = FormBorderStyle.FixedSingle;
            this.MaximizeBox = false;
```

```
            // 1. Cây danh sách lớp bên trái
            trvLop = new TreeView()
            {
                Location = new Point(15, 15),
                Size = new Size(230, 420)
            };
            trvLop.AfterSelect += TrvLop_AfterSelect;

            // 2. Ô chọn lớp bên phải
            lblChonLop = new Label() { Text = "Chọn Lớp", Location = new
Point(270, 20), AutoSize = true };
            cboLop = new ComboBox()
            {
                Location = new Point(350, 17),
                Size = new Size(280, 23),
                DropDownStyle = ComboBoxStyle.DropDownList
            };

            // 3. Khung nhập thông tin sinh viên
            grpTTSV = new GroupBox()
            {
                Text = "Thông tin sinh viên",
                Location = new Point(270, 60),
                Size = new Size(370, 180)
            };

            lblMaSV = new Label() { Text = "Mã SV:", Location = new
Point(20, 30), AutoSize = true };
            txtMaSV = new TextBox() { Location = new Point(90, 27), Size =
new Size(250, 23) };

            lblHoTen = new Label() { Text = "Họ Tên:", Location = new
Point(20, 65), AutoSize = true };
            txtHoTen = new TextBox() { Location = new Point(90, 62), Size =
new Size(250, 23) };

            lblDiaChi = new Label() { Text = "Địa Chỉ:", Location = new
Point(20, 100), AutoSize = true };
            txtDiaChi = new TextBox() { Location = new Point(90, 97), Size =
new Size(250, 23) };

            btnCapNhat = new Button() { Text = "Cập Nhật", Location = new
Point(110, 135), Size = new Size(90, 30) };
            btnXoa = new Button() { Text = "Xóa", Location = new Point(220,
135), Size = new Size(90, 30) };

            btnCapNhat.Click += BtnCapNhat_Click;
            btnXoa.Click += BtnXoa_Click;

            grpTTSV.Controls.AddRange(new Control[] {
```

```
                lblMaSV, txtMaSV, lblHoTen, txtHoTen, lblDiaChi, txtDiaChi,
                btnCapNhat, btnXoa
            });

            // 4. CheckBox để ẩn hoặc hiện khung thêm lớp
            chkThemLop = new CheckBox()
            {
                Text = "Thêm lớp",
                Location = new Point(270, 255),
                AutoSize = true,
                Checked = false
            };
            chkThemLop.CheckedChanged += ChkThemLop_CheckedChanged;

            // 5. Khung thêm lớp
            grpThemLop = new GroupBox()
            {
                Text = "Thông tin lớp",
                Location = new Point(270, 285),
                Size = new Size(370, 100),
                Visible = false
            };

            lblTenLop = new Label() { Text = "Tên lớp:", Location = new
Point(20, 35), AutoSize = true };
            txtTenLop = new TextBox() { Location = new Point(90, 32), Size =
new Size(250, 23) };
            btnThemLop = new Button() { Text = "Thêm lớp", Location = new
Point(240, 62), Size = new Size(100, 28) };
            btnThemLop.Click += BtnThemLop_Click;

            grpThemLop.Controls.AddRange(new Control[] { lblTenLop,
txtTenLop, btnThemLop });

            this.Controls.AddRange(new Control[] {
                trvLop, lblChonLop, cboLop, grpTTSV, chkThemLop, grpThemLop
            });

            this.Load += GiaoDienBai2_Load;
            this.FormClosing += GiaoDienBai2_FormClosing;
        }

        // Khi chạy chương trình: nạp sẵn danh sách lớp mẫu vào TreeView và
ComboBox
        private void GiaoDienBai2_Load(object sender, EventArgs e)
        {
            // Tạo nút gốc
            TreeNode goc = new TreeNode("Danh sách lớp");
            trvLop.Nodes.Add(goc);

            // Danh sách các lớp ban đầu trong tài liệu
```

```
            string[] cacLop = { "05DHTH1", "05DHTH2", "05DHTH3", "05DHTH4"
};

            foreach (string lop in cacLop)
            {
                goc.Nodes.Add(lop); // Thêm vào nhánh của TreeView
                cboLop.Items.Add(lop); // Thêm vào danh sách chọn ComboBox
            }

            goc.Expand(); // Mở rộng nhánh để thấy các lớp
            if (cboLop.Items.Count > 0)
            {
                cboLop.SelectedIndex = 0; // Chọn sẵn lớp đầu tiên
            }
        }

        // Tích chọn thì hiện khung thông tin lớp, bỏ tích thì ẩn đi
        private void ChkThemLop_CheckedChanged(object sender, EventArgs e)
        {
            grpThemLop.Visible = chkThemLop.Checked;
        }

        // Nút Thêm lớp: kiểm tra trùng tên trước khi thêm
        private void BtnThemLop_Click(object sender, EventArgs e)
        {
            string tenLopMoi = txtTenLop.Text.Trim();

            if (tenLopMoi == "")
            {
                MessageBox.Show("Vui lòng nhập tên lớp!", "Thông báo");
                txtTenLop.Focus();
                return;
            }

            // So sánh chuỗi không phân biệt hoa thường để chống trùng lớp
            foreach (var item in cboLop.Items)
            {
                if (string.Compare(item.ToString(), tenLopMoi, true) == 0)
                {
                    MessageBox.Show("Tên lớp này đã tồn tại rồi!", "Báo
lỗi");

                    txtTenLop.Focus();
                    return;
                }
            }

            // Thêm vào nhánh gốc của TreeView và thêm vào ComboBox
            trvLop.Nodes[0].Nodes.Add(tenLopMoi);
            cboLop.Items.Add(tenLopMoi);
            cboLop.SelectedItem = tenLopMoi; // Tự động chọn lớp mới thêm
```

```
            MessageBox.Show("Thêm lớp thành công!");
            txtTenLop.Clear();
        }

        // Nút Cập Nhật: thêm sinh viên vào lớp đang chọn
        private void BtnCapNhat_Click(object sender, EventArgs e)
        {
            // Kiểm tra không được để trống ô nào
            if (txtMaSV.Text.Trim() == "" || txtHoTen.Text.Trim() == "" ||
txtDiaChi.Text.Trim() == "")
            {
                MessageBox.Show("Vui lòng nhập đầy đủ Mã SV, Họ Tên và Địa
Chỉ!", "Thông báo");
                return;
            }

            if (cboLop.SelectedItem == null)
            {
                MessageBox.Show("Chưa chọn lớp nào!", "Thông báo");
                return;
            }

            string maSV = txtMaSV.Text.Trim();
            string hoTen = txtHoTen.Text.Trim();
            string diaChi = txtDiaChi.Text.Trim();
            string lopDangChon = cboLop.SelectedItem.ToString();

            // Kiểm tra không được trùng mã sinh viên trên toàn bộ cây
            if (KiemTraTrungMaSV(trvLop.Nodes[0], maSV))
            {
                MessageBox.Show("Mã sinh viên này đã có rồi, vui lòng đổi mã
khác!", "Báo lỗi");
                txtMaSV.Focus();
                return;
            }

            // Tìm đúng nhánh lớp đang chọn để gắn sinh viên vào
            foreach (TreeNode nodeLop in trvLop.Nodes[0].Nodes)
            {
                if (nodeLop.Text == lopDangChon)
                {
                    // Nút sinh viên có định dạng: "Mã SV, Họ Tên"
                    TreeNode nodeSV = new TreeNode(maSV + ", " + hoTen);
                    nodeSV.Tag = maSV; // Lưu riêng mã SV vào Tag để tiện
kiểm tra và lấy lại

                    // Nút con bên dưới lưu địa chỉ
                    TreeNode nodeDiaChi = new TreeNode(diaChi);
                    nodeSV.Nodes.Add(nodeDiaChi);

                    nodeLop.Nodes.Add(nodeSV);
```

```
                    nodeLop.Expand(); // Mở nhánh ra để nhìn thấy sinh viên
vừa thêm
                    break;
                }
            }

            MessageBox.Show("Thêm sinh viên thành công!");

            // Xóa trắng ô nhập để tiếp tục nhập người khác
            txtMaSV.Clear();
            txtHoTen.Clear();
            txtDiaChi.Clear();
            txtMaSV.Focus();
        }

        // Hàm kiểm tra trùng mã sinh viên bằng cách duyệt các nút
        private bool KiemTraTrungMaSV(TreeNode goc, string maSV)
        {
            foreach (TreeNode n in goc.Nodes)
            {
                // Nếu nút có lưu mã SV mà trùng khớp thì báo true
                if (n.Tag != null && n.Tag.ToString() == maSV)
                {
                    return true;
                }
                // Tìm tiếp các nút con bên dưới
                if (KiemTraTrungMaSV(n, maSV)) return true;
            }
            return false;
        }

        // Khi nhấp chuột vào một nút trên cây: hiển thị thông tin lên các ô
bên phải
        private void TrvLop_AfterSelect(object sender, TreeViewEventArgs e)
        {
            TreeNode nodeDuocChon = e.Node;

            // Chỉ xử lý khi nhấp đúng vào nút sinh viên (có chứa Tag là mã
SV)
            if (nodeDuocChon != null && nodeDuocChon.Tag != null)
            {
                txtMaSV.Text = nodeDuocChon.Tag.ToString();

                // Chuỗi trên nút là "Mã SV, Họ Tên" nên tách chuỗi theo dấu
phẩy
                string[] thongTin = nodeDuocChon.Text.Split(',');
                if (thongTin.Length > 1)
                {
                    txtHoTen.Text = thongTin[1].Trim();
                }
```

```
                // Nút con đầu tiên chính là địa chỉ
                if (nodeDuocChon.Nodes.Count > 0)
                {
                    txtDiaChi.Text = nodeDuocChon.Nodes[0].Text;
                }

                // Chọn đúng lớp của sinh viên này trên ComboBox
                if (nodeDuocChon.Parent != null)
                {
                    cboLop.SelectedItem = nodeDuocChon.Parent.Text;
                }
            }
        }

        // Nút Xóa: chỉ cho xóa nút là sinh viên và hỏi xác nhận trước
        private void BtnXoa_Click(object sender, EventArgs e)
        {
            TreeNode nodeDuocChon = trvLop.SelectedNode;

            // Kiểm tra xem người dùng có đang chọn đúng nút sinh viên không
            if (nodeDuocChon == null || nodeDuocChon.Tag == null)
            {
                MessageBox.Show("Vui lòng chọn một sinh viên trên danh sách
để xóa!", "Thông báo");
                return;
            }

            DialogResult hoi = MessageBox.Show("Bạn có chắc chắn muốn xóa
sinh viên này không?", "Xác nhận", MessageBoxButtons.YesNo,
MessageBoxIcon.Question);
            if (hoi == DialogResult.Yes)
            {
                nodeDuocChon.Remove(); // Xóa khỏi TreeView
                // Làm sạch các ô nhập
                txtMaSV.Clear();
                txtHoTen.Clear();
                txtDiaChi.Clear();
                MessageBox.Show("Đã xóa sinh viên!");
            }
        }

        // Xác nhận trước khi đóng chương trình
        private void GiaoDienBai2_FormClosing(object sender,
FormClosingEventArgs e)
        {
            DialogResult hoi = MessageBox.Show("Bạn có muốn thoát chương
trình không?", "Xác nhận", MessageBoxButtons.YesNo,
MessageBoxIcon.Question);
            if (hoi == DialogResult.No)
            {
                e.Cancel = true;
```

```
            }
        }
    }
}
```

Program.cs

```
using System;
using System.Windows.Forms;

namespace Bai2
{
    static class Program
    {
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new GiaoDienBai2());
        }
    }
}
```

3. Giải thích bài làm

Lớp Bai1.cs:

- Lớp SinhVien dùng để lưu trữ Mã SV, Họ Tên và Địa Chỉ.
- Lớp QuanLyLopHoc quản lý danh sách tên lớp bằng List<string> và ánh xạ sinh viên theo từng lớp bằng Dictionary. Lớp này phụ trách việc kiểm tra trùng tên lớp, trùng mã sinh viên trên toàn trường, cũng như các thao tác thêm/xóa sinh viên.

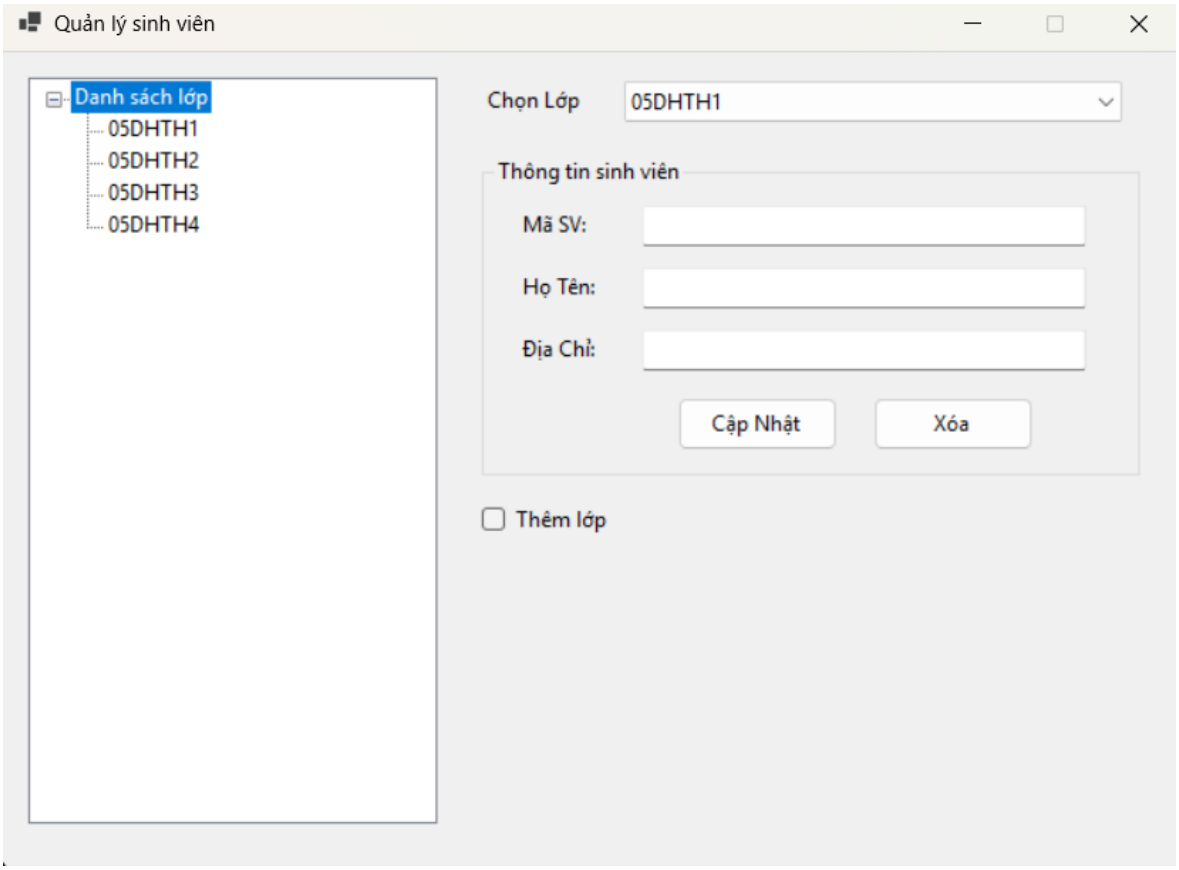
Xử lý trên Form:

- Giao diện chỉ gọi các hàm từ QuanLyLopHoc để lấy dữ liệu nạp vào TreeView và ComboBox.
- Khi chọn ô CheckBox "Thêm lớp", khung nhập lớp sẽ tự động ẩn/hiện bằng thuộc tính Visible.
- Khi nhấn vào một sinh viên trên cây (AfterSelect), thông tin được lấy ra và điền ngược lại vào các ô TextBox bên phải. Nút **Xóa** sẽ hỏi xác nhận yes/no trước khi xóa khỏi cả lớp xử lý lẫn cây giao diện.

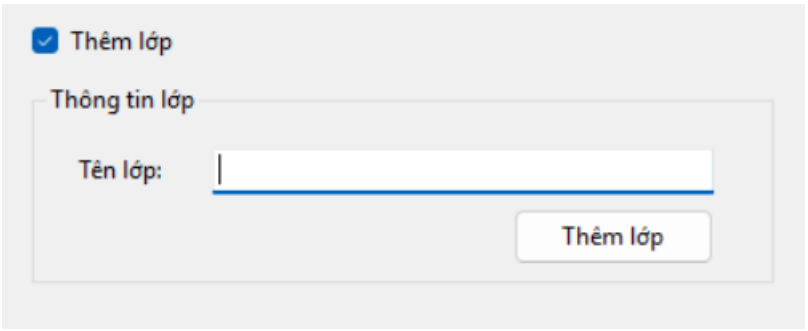
4. Kết quả

Khởi động Form:

Cây danh sách hiển thị nút gốc và mở rộng sẵn 4 lớp

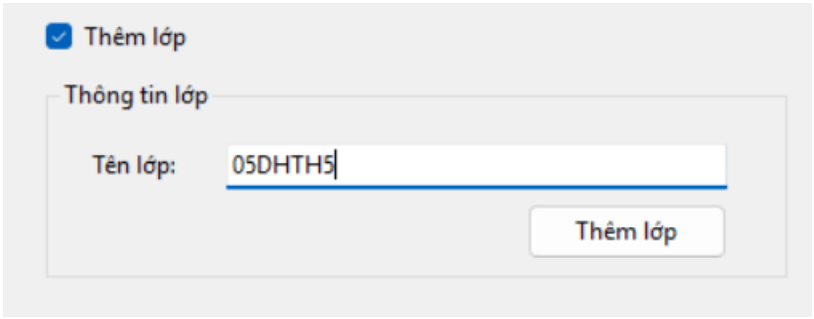


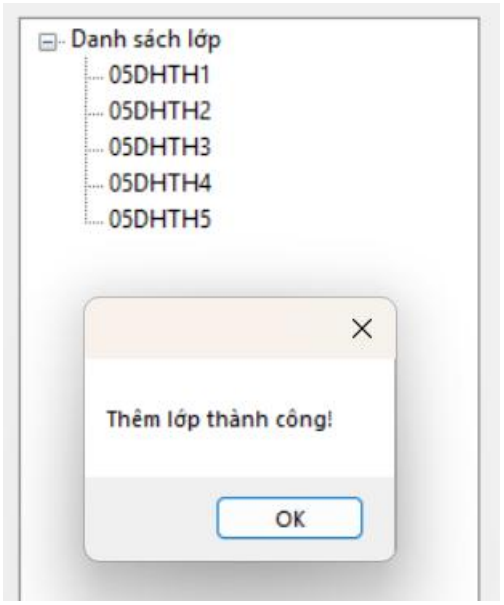
Thêm lớp mới: Tích chọn ô **Thêm lớp** → Khung Thông tin lớp hiện ra

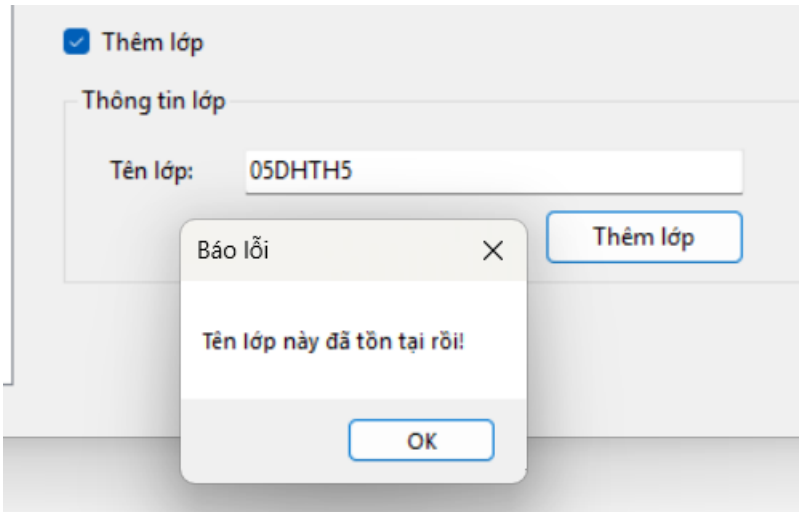


Nhập tên lớp mới VD: 05DHTH5 rồi bấm thêm → Tên lớp mới xuất hiện ở cả TreeView và ComboBox

Nếu nhập lại 05DHTH5 → Thông báo lỗi

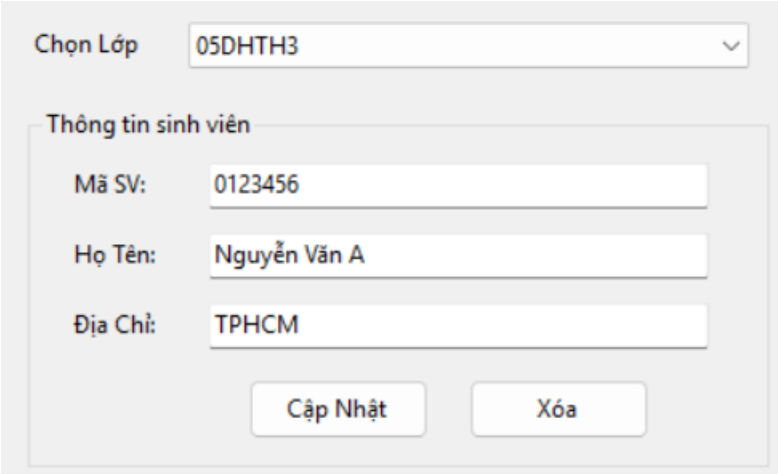


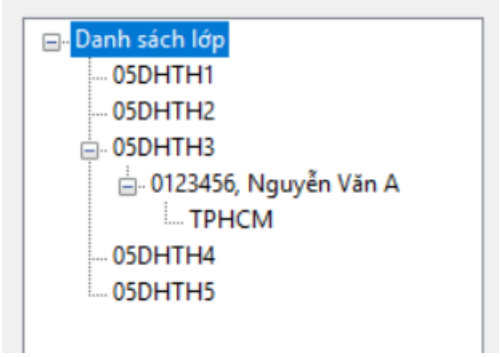




Cập nhật sinh viên vào lớp: Chọn lớp VD: 05DHTH3, nhập MSSV, Họ tên, Địa chỉ, sau đó bấm nút cập nhật → Nhánh 05DHTH3 mở ra nút con

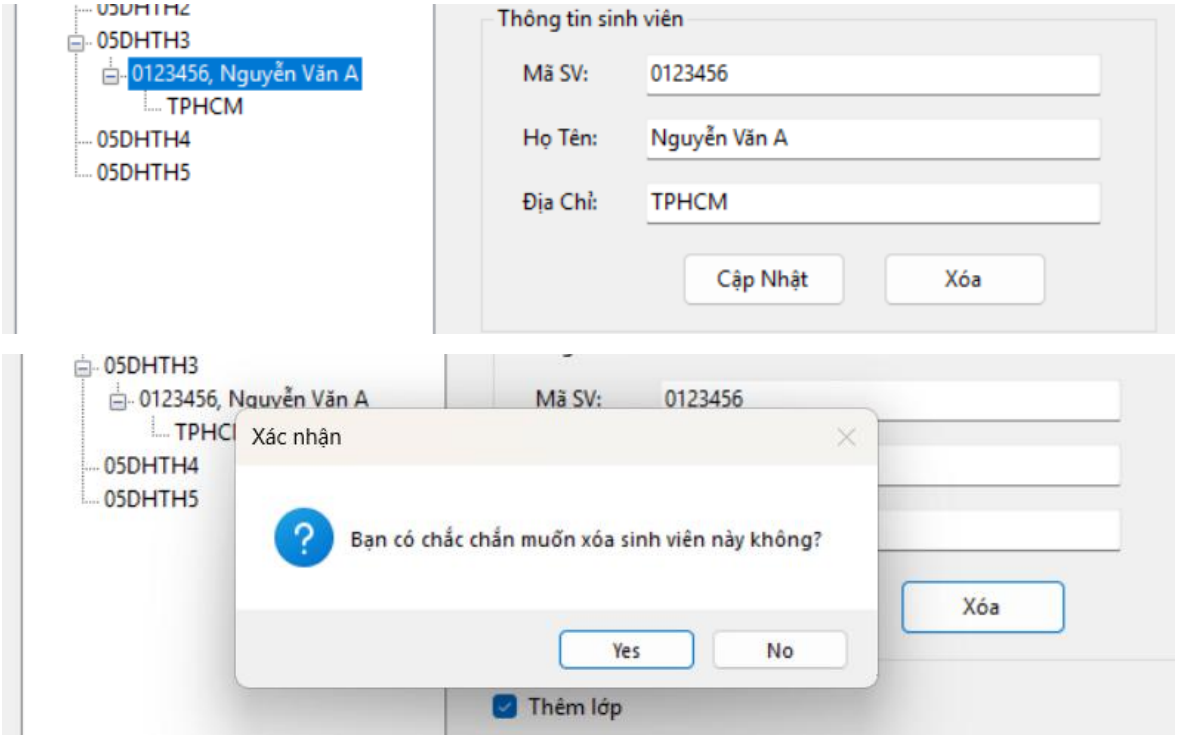
Nếu nhập trùng MSSV → Thông báo lỗi





Chọn xem và xóa sinh viên: Chọn nút sinh viên trên cây → Toàn bộ thông tin tự động đổ lại vào ô bên phải

Bấm nút **Xóa** → Hiện thông báo có chắc chắn muốn xóa sinh viên này, chọn yes/no



Bài tập nâng cao:

1. **Yêu cầu đề**

− Thiết kế giao diện gồm một ListBox và các nút chức năng theo đúng mẫu thiết kế.

− Nút Nhập tên ngẫu nhiên: Sinh ngẫu nhiên 50 họ tên đầy đủ dựa trên 3 mảng HO, TENLOT, TEN rồi nạp vào ListBox.

− Nút Xóa Phần tử đang chọn: Xóa toàn bộ các dòng đang được chọn trong ListBox.

− Nút Xóa phần tử có tên là Sơn: Tìm và xóa tất cả người có tên là "Sơn".

− Nút Xóa Phần tử có họ là Lê: Tìm và xóa tất cả người có họ là "Lê".

−Các nút chuyển kiểu chữ: Chuyển các mục đang chọn thành chữ HOA, chữ thường, hoặc viết Hoa đầu mỗi từ.

− Nút Xóa tất cả các Phần tử: Xóa sạch toàn bộ danh sách trong ListBox.

− Khi nháy đúp chuột (DoubleClick) vào một phần tử: Mở hộp thoại cho phép nhập và cập nhật tên mới vào đúng vị trí đó.

− Hỏi xác nhận trước khi đóng form.

2. **Mã nguồn**

BTNangCao.cs

```
using System;
using System.Collections.Generic;
using System.Globalization;

namespace BTNC
{
    public class XuLyChuoi
    {
        // 3 mảng thành phần họ, chữ lót và tên
        private static readonly string[] MangHo = { "Lê", "Nguyễn", "Lý",
"Trần", "Lâm", "Hồ", "Lai", "Huỳnh", "La" };
        private static readonly string[] MangTenLot = { "Quang", "Thành",
"Ngọc", "Anh", "Xuân", "Bảo", "Cẩm", "Thị", "Kim", "Thái", "Hồng" };
        private static readonly string[] MangTen = { "Hà", "Danh", "Sơn",
"Mai", "Thắng", "Kỳ", "Thành", "Lâm", "Tâm", "Phụng", "Thắm" };

        private static Random rand = new Random();

        // Tạo ngẫu nhiên 50 tên
        public static List<string> Tao50TenNgauNhien()
```

```
        {
            List<string> ds = new List<string>();
            for (int i = 0; i < 50; i++)
            {
                string ho = MangHo[rand.Next(MangHo.Length)];
                string tenLot = MangTenLot[rand.Next(MangTenLot.Length)];
                string ten = MangTen[rand.Next(MangTen.Length)];
                ds.Add(ho + " " + tenLot + " " + ten);
            }
            return ds;
        }

        // Lấy từ đầu tiên (Họ)
        public static string LayHo(string hoTen)
        {
            string[] parts = hoTen.Trim().Split(new char[] { ' ' },
StringSplitOptions.RemoveEmptyEntries);
            return parts.Length > 0 ? parts[0] : "";
        }

        // Lấy từ cuối cùng (Tên)
        public static string LayTen(string hoTen)
        {
            string[] parts = hoTen.Trim().Split(new char[] { ' ' },
StringSplitOptions.RemoveEmptyEntries);
            return parts.Length > 0 ? parts[parts.Length - 1] : "";
        }

        // Chuyển toàn bộ chuỗi sang chữ in HOA
        public static string ChuyenChuHoa(string s)
        {
            return s.ToUpper();
        }

        // Chuyển toàn bộ chuỗi sang chữ thường
        public static string ChuyenChuThuong(string s)
        {
            return s.ToLower();
        }

        // Chuyển sang viết hoa chữ cái đầu mỗi từ
        public static string ChuyenHoaDauMoiTu(string s)
        {
            TextInfo ti = new CultureInfo("vi-VN", false).TextInfo;
            return ti.ToTitleCase(s.ToLower());
        }
    }
}
```

GiaoDienBTNC.cs

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Windows.Forms;

namespace BTNC
{
    public class GiaoDienBTNC : Form
    {
        // Danh sách dữ liệu ngầm để lưu chuỗi thật
        private List<string> dsTen = new List<string>();

        private Button btnNgauNhien;
        private ListBox lstDanhSach;

        // Các nút bấm chức năng bên phải
        private Button btnXoaDangChon;
        private Button btnXoaTenSon;
        private Button btnXoaHoLe;
        private Button btnChuyenHoa;
        private Button btnChuyenThuong;
        private Button btnChuyenHoaDauTu;
        private Button btnXoaTatCa;

        public GiaoDienBTNC()
        {
            KhoiTaoGiaoDien();
        }

        private void KhoiTaoGiaoDien()
        {
            this.Text = "frmChuoi";
            this.Size = new Size(620, 480);
            this.StartPosition = FormStartPosition.CenterScreen;
            this.FormBorderStyle = FormBorderStyle.FixedSingle;
            this.MaximizeBox = false;

            // 1. Nút sinh tên ngẫu nhiên
            btnNgauNhien = new Button()
            {
                Text = "Nhập tên ngẫu nhiên",
                Location = new Point(25, 20),
                Size = new Size(220, 32),
                Font = new Font("Arial", 10, FontStyle.Regular)
            };
            btnNgauNhien.Click += BtnNgauNhien_Click;

            // 2. ListBox hiển thị tên
            lstDanhSach = new ListBox()
```

```
            {
                Location = new Point(25, 60),
                Size = new Size(220, 350),
                SelectionMode = SelectionMode.MultiExtended,
                Font = new Font("Arial", 10, FontStyle.Regular)
            };
            lstDanhSach.DoubleClick += LstDanhSach_DoubleClick;

            // 3. Khởi tạo các nút bên phải
            int startX = 270;
            int startY = 60;
            int btnWidth = 300;
            int btnHeight = 35;
            int cachY = 12;

            btnXoaDangChon = TaoNut("Xóa Phần tử đang chọn", startX, startY,
btnWidth, btnHeight);
            btnXoaTenSon = TaoNut("Xóa phần tử có tên là Sơn", startX,
startY + (btnHeight + cachY) * 1, btnWidth, btnHeight);
            btnXoaHoLe = TaoNut("Xóa Phần tử có họ là Lê", startX, startY +
(btnHeight + cachY) * 2, btnWidth, btnHeight);
            btnChuyenHoa = TaoNut("Chuyển PT đang chọn thành chữ HOA",
startX, startY + (btnHeight + cachY) * 3, btnWidth, btnHeight);
            btnChuyenThuong = TaoNut("Chuyển PT đang chọn thành chữ thường",
startX, startY + (btnHeight + cachY) * 4, btnWidth, btnHeight);
            btnChuyenHoaDauTu = TaoNut("Chuyển PT đang chọn thành viết
Hoa\nđầu mỗi từ", startX, startY + (btnHeight + cachY) * 5, btnWidth,
btnHeight + 5);
            btnXoaTatCa = TaoNut("Xóa tất cả các Phần tử", startX, startY +
(btnHeight + cachY) * 6 + 5, btnWidth, btnHeight);

            // Gắn sự kiện
            btnXoaDangChon.Click += BtnXoaDangChon_Click;
            btnXoaTenSon.Click += BtnXoaTenSon_Click;
            btnXoaHoLe.Click += BtnXoaHoLe_Click;
            btnChuyenHoa.Click += BtnChuyenHoa_Click;
            btnChuyenThuong.Click += BtnChuyenThuong_Click;
            btnChuyenHoaDauTu.Click += BtnChuyenHoaDauTu_Click;
            btnXoaTatCa.Click += BtnXoaTatCa_Click;

            this.Controls.AddRange(new Control[] {
                btnNgauNhien, lstDanhSach,
                btnXoaDangChon, btnXoaTenSon, btnXoaHoLe,
                btnChuyenHoa, btnChuyenThuong, btnChuyenHoaDauTu,
btnXoaTatCa
            });

            this.FormClosing += GiaoDienBTNC_FormClosing;
        }

        private Button TaoNut(string text, int x, int y, int w, int h)
```

```
        {
            return new Button()
            {
                Text = text,
                Location = new Point(x, y),
                Size = new Size(w, h),
                Font = new Font("Arial", 9, FontStyle.Regular)
            };
        }

        // Hàm đổ lại toàn bộ dữ liệu từ dsTen lên ListBox và giữ lại các
dòng đang chọn
        private void CapNhatLaiListBox(List<int> cacDongChon = null)
        {
            lstDanhSach.BeginUpdate();
            lstDanhSach.Items.Clear();

            foreach (string s in dsTen)
            {
                lstDanhSach.Items.Add(s);
            }

            // Chọn lại các dòng cũ nếu có
            if (cacDongChon != null)
            {
                foreach (int idx in cacDongChon)
                {
                    if (idx < lstDanhSach.Items.Count)
                    {
                        lstDanhSach.SetSelected(idx, true);
                    }
                }
            }

            lstDanhSach.EndUpdate();
        }

        // Nút Nhập tên ngẫu nhiên
        private void BtnNgauNhien_Click(object sender, EventArgs e)
        {
            dsTen = XuLyChuoi.Tao50TenNgauNhien();
            CapNhatLaiListBox();
        }

        // Nút Xóa Phần tử đang chọn
        private void BtnXoaDangChon_Click(object sender, EventArgs e)
        {
            if (lstDanhSach.SelectedIndices.Count == 0)
            {
                MessageBox.Show("Vui lòng chọn ít nhất một phần tử để xóa!",
"Thông báo");
```

```
                return;
            }

            // Lấy danh sách index đang chọn rồi duyệt ngược để xóa khỏi
dsTen
            List<int> viTri = new List<int>();
            foreach (int i in lstDanhSach.SelectedIndices) viTri.Add(i);
            viTri.Sort();

            for (int i = viTri.Count - 1; i >= 0; i--)
            {
                dsTen.RemoveAt(viTri[i]);
            }

            CapNhatLaiListBox();
        }

        // Nút Xóa phần tử có tên là Sơn
        private void BtnXoaTenSon_Click(object sender, EventArgs e)
        {
            for (int i = dsTen.Count - 1; i >= 0; i--)
            {
                if (string.Compare(XuLyChuoi.LayTen(dsTen[i]), "Sơn", true)
== 0)
                {
                    dsTen.RemoveAt(i);
                }
            }
            CapNhatLaiListBox();
        }

        // Nút Xóa Phần tử có họ là Lê
        private void BtnXoaHoLe_Click(object sender, EventArgs e)
        {
            for (int i = dsTen.Count - 1; i >= 0; i--)
            {
                if (string.Compare(XuLyChuoi.LayHo(dsTen[i]), "Lê", true) ==
0)
                {
                    dsTen.RemoveAt(i);
                }
            }
            CapNhatLaiListBox();
        }

        // Nút Chuyển PT đang chọn thành chữ HOA (Chắc chắn hoạt động 100%)
        private void BtnChuyenHoa_Click(object sender, EventArgs e)
        {
            if (lstDanhSach.SelectedIndices.Count == 0)
            {
```

```
                MessageBox.Show("Vui lòng chọn phần tử cần chuyển đổi!",
"Thông báo");
                return;
            }

            List<int> viTri = new List<int>();
            foreach (int i in lstDanhSach.SelectedIndices)
            {
                viTri.Add(i);
                dsTen[i] = XuLyChuoi.ChuyenChuHoa(dsTen[i]);
            }

            CapNhatLaiListBox(viTri);
        }

        // Nút Chuyển PT đang chọn thành chữ thường
        private void BtnChuyenThuong_Click(object sender, EventArgs e)
        {
            if (lstDanhSach.SelectedIndices.Count == 0)
            {
                MessageBox.Show("Vui lòng chọn phần tử cần chuyển đổi!",
"Thông báo");
                return;
            }

            List<int> viTri = new List<int>();
            foreach (int i in lstDanhSach.SelectedIndices)
            {
                viTri.Add(i);
                dsTen[i] = XuLyChuoi.ChuyenChuThuong(dsTen[i]);
            }

            CapNhatLaiListBox(viTri);
        }

        // Nút Chuyển PT đang chọn thành viết Hoa đầu mỗi từ
        private void BtnChuyenHoaDauTu_Click(object sender, EventArgs e)
        {
            if (lstDanhSach.SelectedIndices.Count == 0)
            {
                MessageBox.Show("Vui lòng chọn phần tử cần chuyển đổi!",
"Thông báo");
                return;
            }

            List<int> viTri = new List<int>();
            foreach (int i in lstDanhSach.SelectedIndices)
            {
                viTri.Add(i);
                dsTen[i] = XuLyChuoi.ChuyenHoaDauMoiTu(dsTen[i]);
            }
```

```
            CapNhatLaiListBox(viTri);
        }

        // Nút Xóa tất cả các Phần tử
        private void BtnXoaTatCa_Click(object sender, EventArgs e)
        {
            dsTen.Clear();
            CapNhatLaiListBox();
        }

        // Nhấp đúp chuột vào phần tử để đổi tên
        private void LstDanhSach_DoubleClick(object sender, EventArgs e)
        {
            if (lstDanhSach.SelectedIndex != -1)
            {
                int viTri = lstDanhSach.SelectedIndex;
                string tenCu = dsTen[viTri];

                Form hopThoai = new Form()
                {
                    Text = "Đổi tên mới",
                    Size = new Size(350, 150),
                    StartPosition = FormStartPosition.CenterParent,
                    FormBorderStyle = FormBorderStyle.FixedDialog,
                    MaximizeBox = false,
                    MinimizeBox = false
                };

                Label lblPrompt = new Label() { Text = "Nhập tên mới:",
Location = new Point(20, 15), AutoSize = true };
                TextBox txtMoi = new TextBox() { Text = tenCu, Location =
new Point(20, 38), Width = 290 };
                Button btnOK = new Button() { Text = "OK", Location = new
Point(140, 72), Width = 80, DialogResult = DialogResult.OK };
                Button btnHuy = new Button() { Text = "Hủy", Location = new
Point(230, 72), Width = 80, DialogResult = DialogResult.Cancel };

                hopThoai.Controls.AddRange(new Control[] { lblPrompt,
txtMoi, btnOK, btnHuy });
                hopThoai.AcceptButton = btnOK;

                if (hopThoai.ShowDialog() == DialogResult.OK)
                {
                    if (!string.IsNullOrWhiteSpace(txtMoi.Text))
                    {
                        dsTen[viTri] = txtMoi.Text.Trim();
                        CapNhatLaiListBox();
                    }
                }
            }
```

```
        }

        // Xác nhận trước khi đóng form
        private void GiaoDienBTNC_FormClosing(object sender,
FormClosingEventArgs e)
        {
            DialogResult hoi = MessageBox.Show("Bạn có muốn thoát chương
trình không?", "Xác nhận", MessageBoxButtons.YesNo,
MessageBoxIcon.Question);
            if (hoi == DialogResult.No)
            {
                e.Cancel = true;
            }
        }
    }
}
```

Program.cs

```
using System;
using System.Windows.Forms;

namespace BTNC
{
    static class Program
    {
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new GiaoDienBTNC());
        }
    }
}
```

3. Giải thích bài làm

Khai báo các mảng hằng chứa họ, chữ lót, tên để sinh ngẫu nhiên 50 người bằng Random. Hàm LayHo và LayTen bóc tách chuỗi bằng Split(' ') để lấy phần tử đầu tiên và cuối cùng. Việc viết hoa đầu mỗi từ được chuyển trước về .ToLower() rồi mới qua ToTitleCase() để xử lý chuẩn xác tiếng Việt có dấu.

Đồng bộ dữ liệu bằng List<string> dsTen: Thay vì thao tác trực tiếp lên ListBox.Items (dễ bị lỗi không cập nhật chữ và mất trạng thái bôi đen), toàn bộ dữ liệu gốc được lưu trong dsTen. Khi chuyển kiểu chữ, chương trình cập nhật

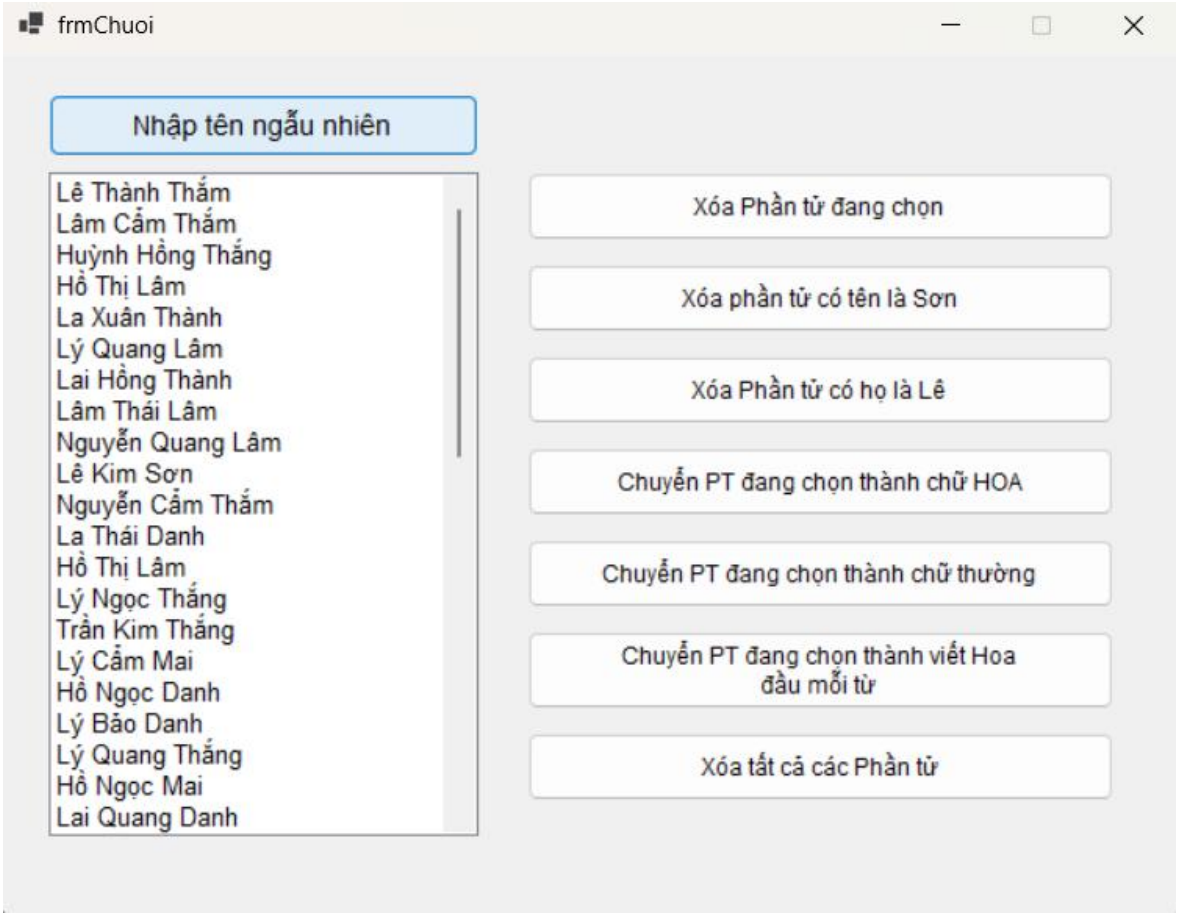
giá trị trong dsTen, sau đó nạp lại ListBox qua BeginUpdate() / EndUpdate() và dùng SetSelected để giữ nguyên trạng thái chọn của người dùng.

Xóa ngược từ cuối về đầu: Khi xóa theo điều kiện (tên "Sơn", họ "Lê" hoặc các dòng đang chọn), vòng lặp duyệt từ dsTen.Count - 1 về 0 để chỉ số của các dòng phía trước không bị xáo trộn khi xóa.

Hộp thoại sửa tên: Bắt sự kiện DoubleClick của ListBox. Khi nhấp đúp vào dòng nào, một Form nhỏ xuất hiện lấy tên cũ vào ô nhập; người dùng sửa lại và bấm OK thì chuỗi mới lập tức được lưu vào danh sách và cập nhật ra màn hình

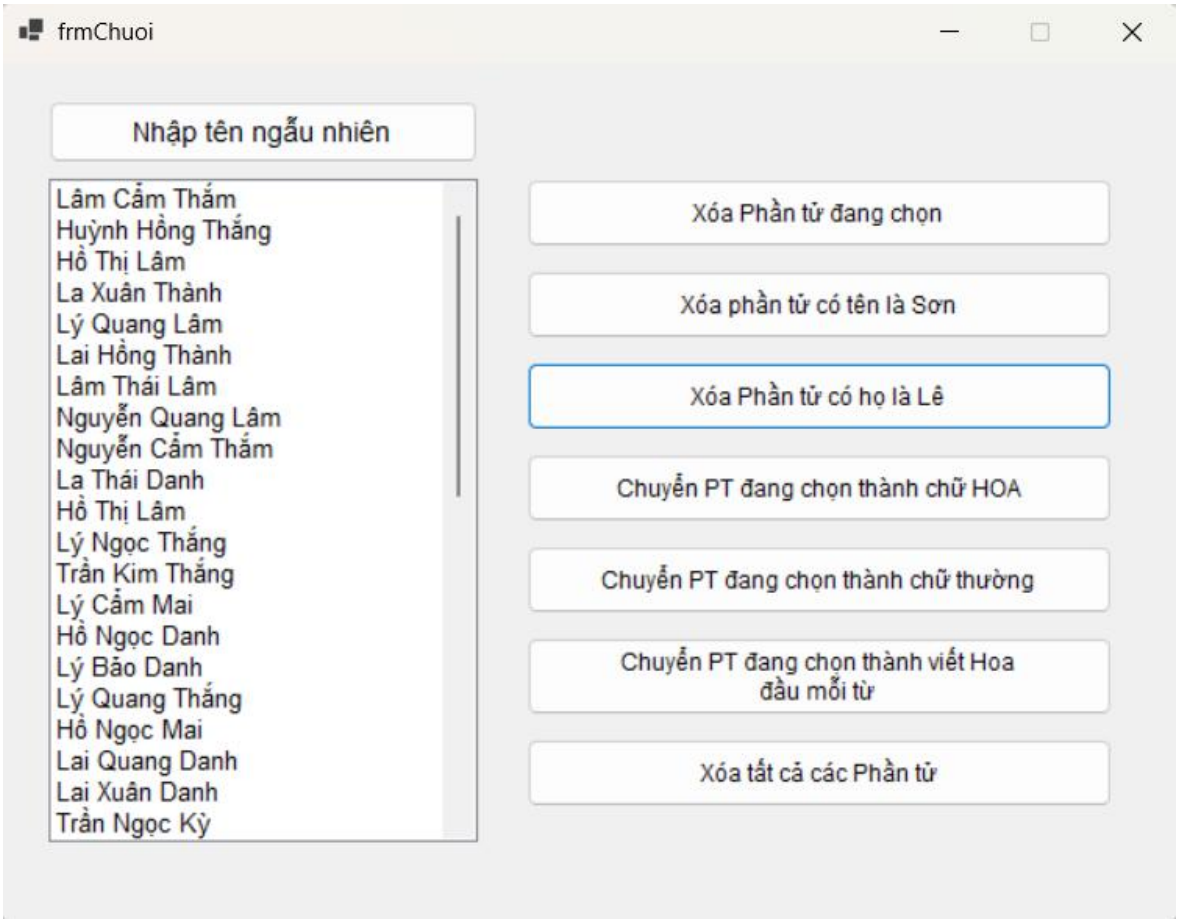
4. Kết quả

Bấm nút **Nhập tên ngẫu nhiên** → Hiển thị 50 tên ngẫu nhiên được ghép từ 3 mảng



Xóa theo họ và tên: Bấm nút **Xóa phần tử có tên là Sơn** → Toàn bộ người có tên cuối cùng là “Sơn” bị xóa khỏi danh sách

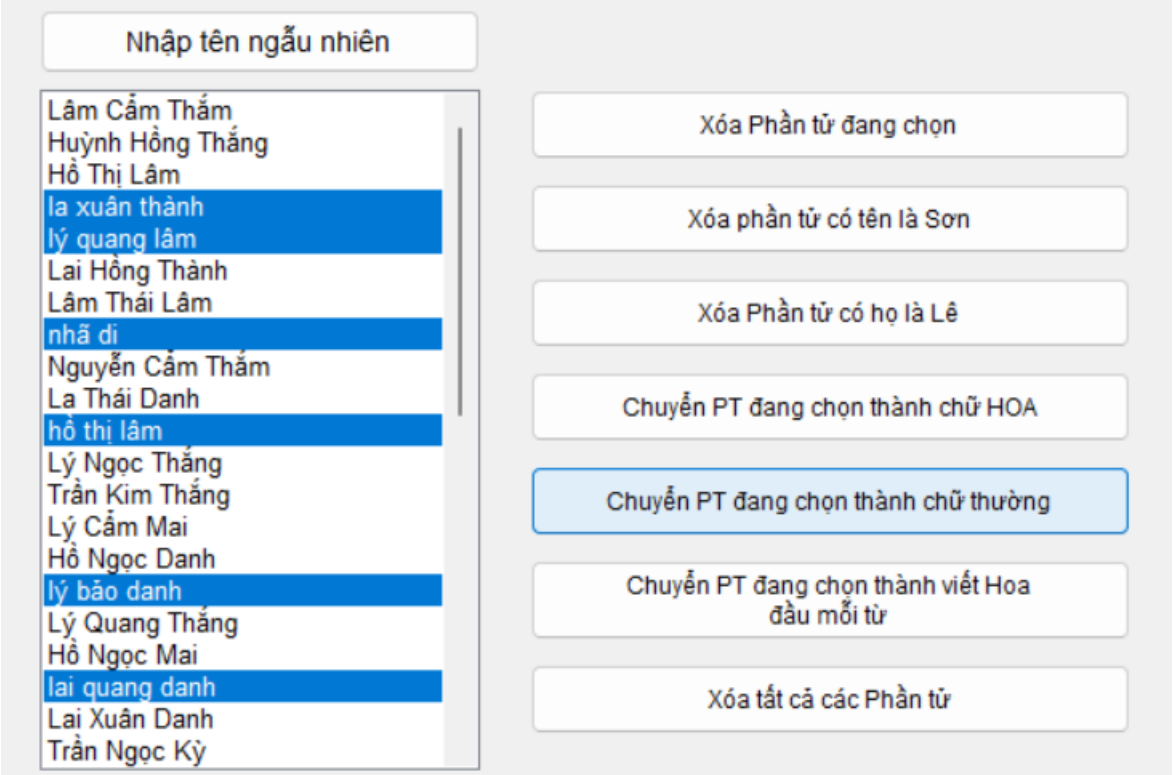
Bấm nút **Xóa phần tử có họ là Lê** → Toàn bộ người có họ "Lê" bị xóa khỏi danh sách



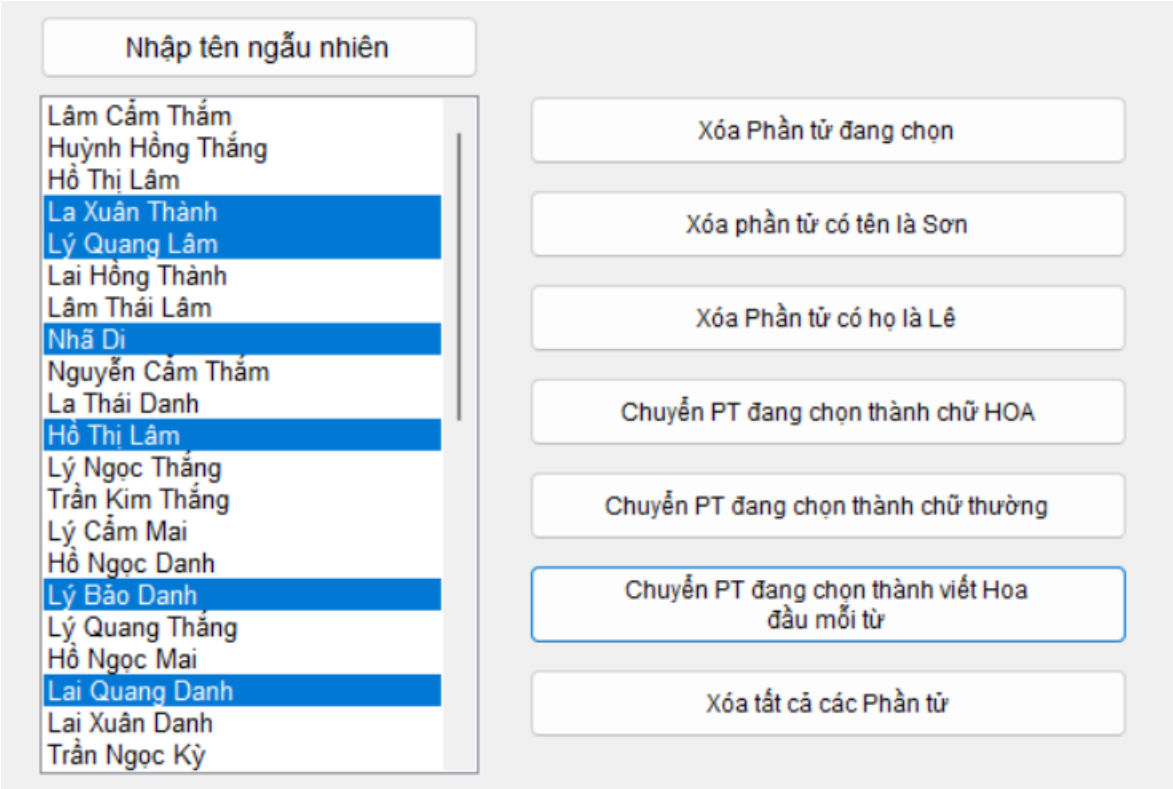
Chuyển đổi kiểu chữ: Bấm **Chuyển PT đang chọn thành chữ HOA** → Các dòng được chọn đổi thành in hoa toàn bộ



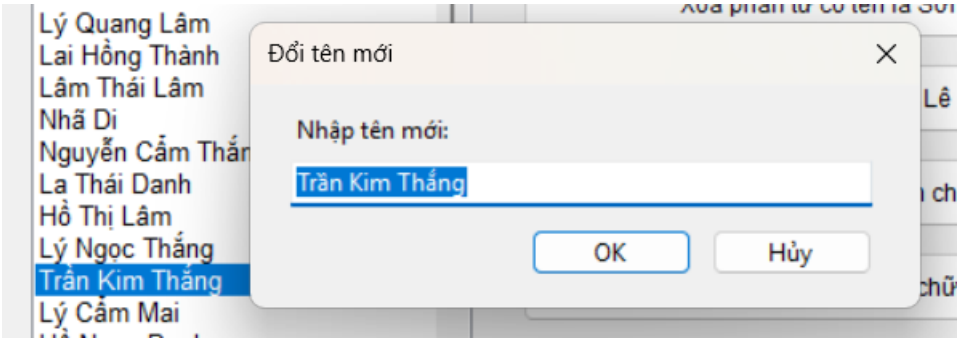
Bấm **Chuyển PT đang chọn thành chữ thường** → Các dòng được chọn đổi thành in thường toàn bộ

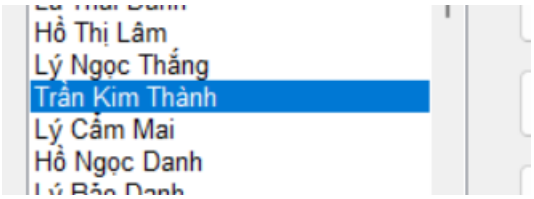


Bấm **Chuyển PT đang chọn thành viết Hoa đầu mỗi từ** → Các dòng trở về đúng định dạng chữ cái đầu viết hoa



Nhấp đúp đổi tên: Nhấp đúp chuột vào một dòng bất kỳ → Hiện lên ô với tên hiện tại → Sửa tên thành tên mới bấm **OK** → ListBox đổi theo tên mới vừa nhập





Bài tập về nhà:

Bài 1:

1. **Yêu cầu đề**

− Thiết kế giao diện ứng dụng từ điển gồm 2 tab: ANH - VIỆT và VIỆT - ANH.

− Khi nhập từ cần tra vào ComboBox, chương trình sẽ tự động dò tìm đến từ khớp với ký tự gần nhất.

− Khi nhấn phím Enter tại ô tra cứu hoặc nháy đúp chuột (DoubleClick) vào từ trong ListBox, nghĩa tương ứng của từ sẽ hiển thị sang TextBox bên phải.

− Danh sách các từ vựng được lưu sẵn vào List.

− Nút Thoát: Thoát khỏi chương trình và có hộp thoại xác nhận trước khi đóng

2. **Mã nguồn**

Bai1VN.cs

```
using System;
using System.Collections.Generic;

namespace BTVN1
{
    public class MucTuDien
    {
        public string TiengAnh { get; set; }
        public string TiengViet { get; set; }

        public MucTuDien(string tiengAnh, string tiengViet)
        {
            TiengAnh = tiengAnh;
            TiengViet = tiengViet;
        }
    }

    // Lớp quản lý tra cứu từ điển
    public class TuDien
    {
        // Danh sách từ lưu sẵn vào List theo yêu cầu đề bài
        private List<MucTuDien> danhSachTu = new List<MucTuDien>();

        public TuDien()
        {
            NapTuVungMacDinh();
        }

        // Nạp sẵn một số từ vựng mẫu như trong hình tài liệu
        private void NapTuVungMacDinh()
```

```
        {
            danhSachTu.Add(new MucTuDien("student", "Sinh viên, học sinh"));
            danhSachTu.Add(new MucTuDien("teacher", "Giáo viên, thầy cô"));
            danhSachTu.Add(new MucTuDien("worker", "Công nhân, người lao
động"));
            danhSachTu.Add(new MucTuDien("hat", "Cái mũ, cái nón"));
            danhSachTu.Add(new MucTuDien("head", "Cái đầu"));
            danhSachTu.Add(new MucTuDien("mouse", "Con chuột"));
            danhSachTu.Add(new MucTuDien("cat", "Con mèo"));
            danhSachTu.Add(new MucTuDien("dog", "Con chó"));
            danhSachTu.Add(new MucTuDien("snake", "Con rắn"));
            danhSachTu.Add(new MucTuDien("lamb", "Cừu con"));
            danhSachTu.Add(new MucTuDien("loan", "Khoản vay"));
            danhSachTu.Add(new MucTuDien("frog", "Con ếch"));
            danhSachTu.Add(new MucTuDien("sheep", "Con cừu"));
        }

        // Lấy toàn bộ danh sách từ
        public List<MucTuDien> LayDanhSach() => danhSachTu;

        // Tra cứu nghĩa Anh sang Việt
        public string TraAnhViet(string tuAnh)
        {
            foreach (var item in danhSachTu)
            {
                if (string.Compare(item.TiengAnh, tuAnh.Trim(), true) == 0)
                {
                    return item.TiengViet;
                }
            }
            return "Không tìm thấy từ này trong từ điển!";
        }

        // Tra cứu nghĩa Việt sang Anh
        public string TraVietAnh(string tuViet)
        {
            foreach (var item in danhSachTu)
            {
                if (string.Compare(item.TiengViet, tuViet.Trim(), true) ==
0)
                {
                    return item.TiengAnh;
                }
            }
            return "Không tìm thấy từ này trong từ điển!";
        }
    }
}
```

GiaoDienB1VN.cs

```
using System;
using System.Drawing;
using System.Windows.Forms;

namespace BTVN1
{
    public class GiaoDienB1VN : Form
    {
        private TuDien tuDien = new TuDien();

        private TabControl tabTuDien;
        private TabPage tabAnhViet, tabVietAnh;

        // Tab Anh - Viet
        private ComboBox cboAnh;
        private ListBox lstAnh;
        private TextBox txtNghiaViet;

        // Tab Viet - Anh
        private ComboBox cboViet;
        private ListBox lstViet;
        private TextBox txtNghiaAnh;

        private Button btnThoat;

        public GiaoDienB1VN()
        {
            KhoiTaoGiaoDien();
        }

        private void KhoiTaoGiaoDien()
        {
            this.Text = "TỪ ĐIỂN ANH VIỆT - VIỆT ANH";
            this.Size = new Size(580, 480);
            this.StartPosition = FormStartPosition.CenterScreen;
            this.FormBorderStyle = FormBorderStyle.FixedDialog;
            this.MaximizeBox = false;

            // 1. TabControl
            tabTuDien = new TabControl()
            {
                Location = new Point(20, 15),
                Size = new Size(525, 370)
            };

            tabAnhViet = new TabPage("ANH - VIỆT");
            tabVietAnh = new TabPage("VIỆT - ANH");

            KhoiTaoTabAnhViet();
```

```
            KhoiTaoTabVietAnh();

            tabTuDien.TabPages.Add(tabAnhViet);
            tabTuDien.TabPages.Add(tabVietAnh);

            // 2. Nut Thoat
            btnThoat = new Button()
            {
                Text = "Thoát",
                Location = new Point(430, 395),
                Size = new Size(115, 32),
                Font = new Font("Arial", 10, FontStyle.Regular)
            };
            btnThoat.Click += (s, e) => this.Close();

            this.Controls.Add(tabTuDien);
            this.Controls.Add(btnThoat);

            // Gan su kien Load va Closing
            this.Load += GiaoDienB1VN_Load;
            this.FormClosing += GiaoDienB1VN_FormClosing;
        }

        private void KhoiTaoTabAnhViet()
        {
            Label lblAnh = new Label() { Text = "Tiếng Anh", Location = new
Point(20, 15), AutoSize = true };
            Label lblViet = new Label() { Text = "Tiếng Việt", Location =
new Point(260, 15), AutoSize = true };

            cboAnh = new ComboBox()
            {
                Location = new Point(20, 40),
                Size = new Size(210, 25)
            };
            cboAnh.KeyDown += CboAnh_KeyDown;

            lstAnh = new ListBox()
            {
                Location = new Point(20, 72),
                Size = new Size(210, 245),
                Font = new Font("Arial", 10, FontStyle.Regular)
            };
            lstAnh.SelectedIndexChanged += LstAnh_SelectedIndexChanged;
            lstAnh.DoubleClick += (s, e) => TraTuAnhViet();

            txtNghiaViet = new TextBox()
            {
                Location = new Point(260, 40),
                Size = new Size(235, 277),
                Multiline = true,
```

```
                ReadOnly = true,
                BackColor = Color.White,
                Font = new Font("Arial", 10, FontStyle.Regular)
            };

            tabAnhViet.Controls.AddRange(new Control[] { lblAnh, lblViet,
cboAnh, lstAnh, txtNghiaViet });
        }

        private void KhoiTaoTabVietAnh()
        {
            Label lblViet = new Label() { Text = "Tiếng Việt", Location =
new Point(20, 15), AutoSize = true };
            Label lblAnh = new Label() { Text = "Tiếng Anh", Location = new
Point(260, 15), AutoSize = true };

            cboViet = new ComboBox()
            {
                Location = new Point(20, 40),
                Size = new Size(210, 25)
            };
            cboViet.KeyDown += CboViet_KeyDown;

            lstViet = new ListBox()
            {
                Location = new Point(20, 72),
                Size = new Size(210, 245),
                Font = new Font("Arial", 10, FontStyle.Regular)
            };
            lstViet.SelectedIndexChanged += LstViet_SelectedIndexChanged;
            lstViet.DoubleClick += (s, e) => TraTuVietAnh();

            txtNghiaAnh = new TextBox()
            {
                Location = new Point(260, 40),
                Size = new Size(235, 277),
                Multiline = true,
                ReadOnly = true,
                BackColor = Color.White,
                Font = new Font("Arial", 10, FontStyle.Regular)
            };

            tabVietAnh.Controls.AddRange(new Control[] { lblViet, lblAnh,
cboViet, lstViet, txtNghiaAnh });
        }

        // Nap du lieu khi form da san sang hien thi (An toan tuyet doi)
        private void GiaoDienB1VN_Load(object sender, EventArgs e)
        {
            foreach (var item in tuDien.LayDanhSach())
            {
```

```
                cboAnh.Items.Add(item.TiengAnh);
                lstAnh.Items.Add(item.TiengAnh);

                cboViet.Items.Add(item.TiengViet);
                lstViet.Items.Add(item.TiengViet);
            }

            // Bat che do goi y sau khi da co du lieu
            cboAnh.AutoCompleteMode = AutoCompleteMode.SuggestAppend;
            cboAnh.AutoCompleteSource = AutoCompleteSource.ListItems;

            cboViet.AutoCompleteMode = AutoCompleteMode.SuggestAppend;
            cboViet.AutoCompleteSource = AutoCompleteSource.ListItems;
        }

        private void LstAnh_SelectedIndexChanged(object sender, EventArgs e)
        {
            if (lstAnh.SelectedItem != null)
            {
                cboAnh.Text = lstAnh.SelectedItem.ToString();
            }
        }

        private void LstViet_SelectedIndexChanged(object sender, EventArgs
e)
        {
            if (lstViet.SelectedItem != null)
            {
                cboViet.Text = lstViet.SelectedItem.ToString();
            }
        }

        private void CboAnh_KeyDown(object sender, KeyEventArgs e)
        {
            if (e.KeyCode == Keys.Enter)
            {
                TraTuAnhViet();
                e.SuppressKeyPress = true;
            }
        }

        private void CboViet_KeyDown(object sender, KeyEventArgs e)
        {
            if (e.KeyCode == Keys.Enter)
            {
                TraTuVietAnh();
                e.SuppressKeyPress = true;
            }
        }

        private void TraTuAnhViet()
```

```
        {
            string tu = cboAnh.Text.Trim();
            if (string.IsNullOrEmpty(tu)) return;

            txtNghiaViet.Text = tuDien.TraAnhViet(tu);

            int idx = lstAnh.FindStringExact(tu);
            if (idx != -1) lstAnh.SelectedIndex = idx;
        }

        private void TraTuVietAnh()
        {
            string tu = cboViet.Text.Trim();
            if (string.IsNullOrEmpty(tu)) return;

            txtNghiaAnh.Text = tuDien.TraVietAnh(tu);

            int idx = lstViet.FindStringExact(tu);
            if (idx != -1) lstViet.SelectedIndex = idx;
        }

        private void GiaoDienB1VN_FormClosing(object sender,
FormClosingEventArgs e)
        {
            DialogResult hoi = MessageBox.Show("Bạn có muốn thoát chương
trình từ điển không?", "Xác nhận", MessageBoxButtons.YesNo,
MessageBoxIcon.Question);
            if (hoi == DialogResult.No)
            {
                e.Cancel = true;
            }
        }
    }
}
```

Program.cs

```
using System;
using System.Windows.Forms;

namespace BTVN1
{
    static class Program
    {
        static void Main()
        {
            try
            {
                Application.EnableVisualStyles();
                Application.SetCompatibleTextRenderingDefault(false);
```

```
                Application.Run(new GiaoDienB1VN());
            }
            catch (Exception ex)
            {
                MessageBox.Show("Lỗi khởi chạy: " + ex.Message, "Lỗi hệ
thống");
            }
        }
    }
}
```

3. Giải thích bài làm

Lớp MucTuDien đại diện cho một cặp từ tương ứng TiengAnh và TiengViet. Lớp TuDien quản lý tập dữ liệu bằng danh sách List<MucTuDien> và cung cấp hai phương thức tra cứu độc lập: TraAnhViet(string) và TraVietAnh(string). Sử dụng hàm string.Compare(..., true) giúp việc so sánh từ không bị phân biệt chữ in hoa hay chữ thường.

Phân trang bằng TabControl: Ứng dụng tạo 2 Tab riêng biệt (ANH - VIỆT và VIỆT - ANH) trên cùng một cửa sổ giúp giao diện gọn gàng, người dùng dễ chuyển đổi chiều tra cứu mà không cần mở nhiều form.

Tự động gợi ý từ: Thiết lập AutoCompleteMode = AutoCompleteMode.SuggestAppend và nguồn AutoCompleteSource = AutoCompleteSource.ListItems cho cả hai ComboBox trong sự kiện Form_Load. Nhờ vậy, ngay khi người dùng gõ từng ký tự đầu, ComboBox tự động khớp đến từ vựng gần nhất trong danh sách theo đúng yêu cầu đề bài.

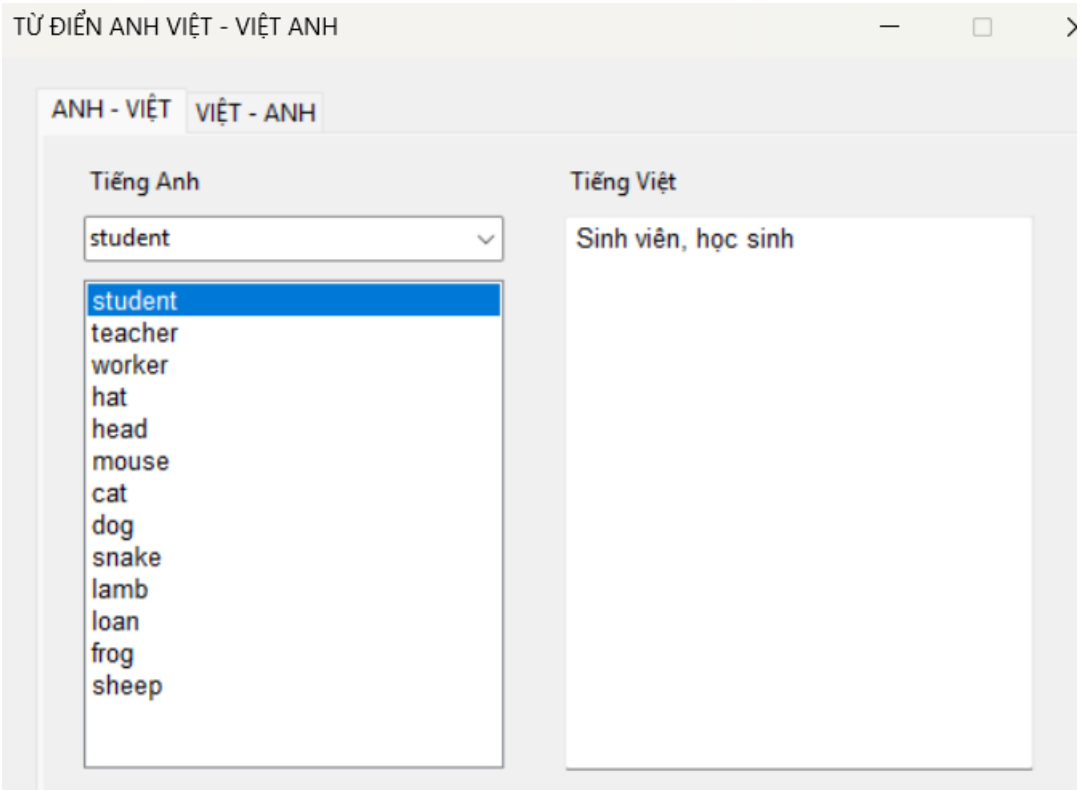
Xử lý sự kiện tra từ linh hoạt:

- Khi gõ xong từ trong ô ComboBox và nhấn phím **Enter** (e.KeyCode == Keys.Enter), hàm tra từ sẽ đọc chuỗi, gọi phương thức nghiệp vụ và in kết quả nghĩa sang ô TextBox bên phải.
- Khi người dùng nháy đúp chuột (DoubleClick) vào một từ trong ListBox, chương trình cũng kích hoạt hàm tra từ tương ứng và đồng bộ chữ lên ComboBox.

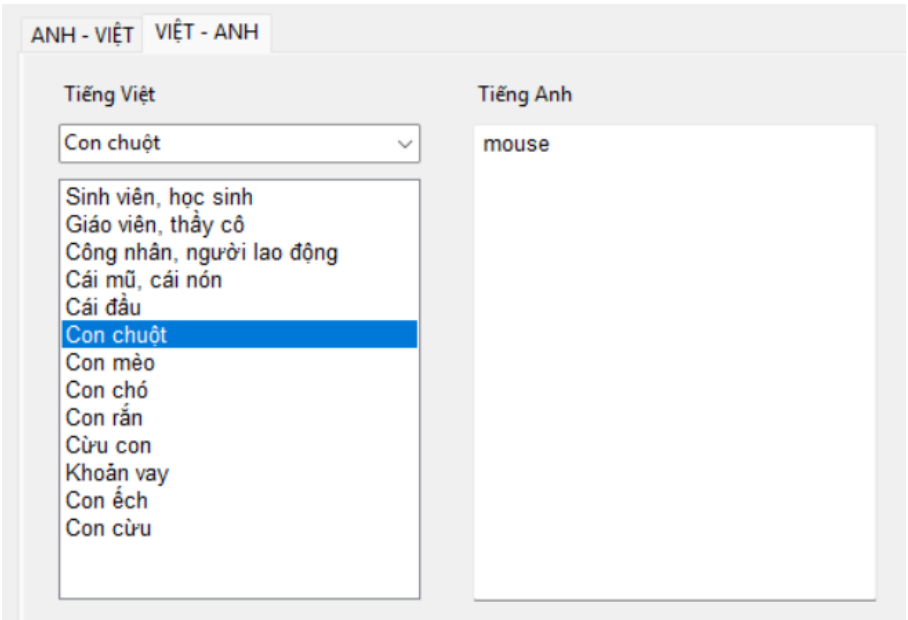
4. Kết quả

Gõ tìm từ và nhấn Enter ở tab ANH – VIỆT: Tại ô Tiếng Anh gõ ký tự student → Combobox tự động xổ đến từ student, nhấn Enter → ô Tiếng Việt hiển thị: "Sinh viên, học sinh"

Nhấn đúp chuột vào bất kỳ từ Tiếng Anh nào → chữ Tiếng Việt tương ứng sẽ xuất hiện ở ô Tiếng Việt



Ngược lại tương tự ở tab VIỆT – ANH



Bài 2:

1. **Yêu cầu đề**

Nhập các số tự nhiên từ ô TextBox và thêm vào ListBox (có kiểm tra dữ liệu hợp lệ). Form ban đầu mở lên để trống hoàn toàn để người dùng tự nhập.

Các nút chức năng thực hiện các yêu cầu:

- Tổng các phần tử trong List: Tính tổng toàn bộ phần tử và hiển thị thông báo lên MessageBox.
- Xóa Phần tử đầu và cuối: Xóa phần tử đầu tiên và phần tử cuối cùng trong ListBox.
- Xóa Phần tử đang chọn: Xóa toàn bộ các số đang được chọn trong ListBox.
- Tăng mỗi phần tử lên 2: Cộng thêm 2 đơn vị vào từng phần tử trong danh sách.
- Thay bằng bình phương: Gán mỗi phần tử bằng bình phương của chính nó.
- Chọn số chẵn: Tự động quét và chọn tất cả các số chẵn trong ListBox.
- Chọn số lẻ: Tự động quét và chọn tất cả các số lẻ trong ListBox.

Nút KẾT THÚC: Đóng chương trình có hộp thoại xác nhận từ người dùng.

2. **Mã nguồn**

Bai2VN.cs

```
using System;
using System.Collections.Generic;

namespace BTVN2
{
    public class XuLySoNguyen
    {
        // Tính tổng các phần tử trong danh sách
        public static long TinhTong(List<int> danhSach)
        {
            long tong = 0;
            foreach (int x in danhSach)
            {
                tong += x;
            }
            return tong;
        }

        // Tăng mỗi phần tử trong danh sách lên 2
        public static void TangLenHai(List<int> danhSach)
```

```
        {
            for (int i = 0; i < danhSach.Count; i++)
            {
                danhSach[i] += 2;
            }
        }

        // Thay mỗi phần tử bằng bình phương của chính nó
        public static void ThayBangBinhPhuong(List<int> danhSach)
        {
            for (int i = 0; i < danhSach.Count; i++)
            {
                danhSach[i] = danhSach[i] * danhSach[i];
            }
        }

        // Kiểm tra số chẵn
        public static bool LaSoChan(int n)
        {
            return n % 2 == 0;
        }

        // Kiểm tra số lẻ
        public static bool LaSoLe(int n)
        {
            return n % 2 != 0;
        }
    }
}
```

GiaoDienB2VN.cs

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Windows.Forms;

namespace BTVN2
{
    public class GiaoDienB2VN : Form
    {
        private List<int> dsSo = new List<int>();

        // Tiêu đề và nhóm nhập liệu bên trái
        private Label lblTieuDe;
        private Label lblListBox;
        private TextBox txtNhap;
        private Button btnNhap;
        private ListBox lstSo;
```

```
        // Nhóm nút xử lý bên phải
        private Label lblXuLy;
        private Button btnTong;
        private Button btnXoaDauCuoi;
        private Button btnXoaDangChon;
        private Button btnTang2;
        private Button btnBinhPhuong;
        private Button btnChonChan;
        private Button btnChonLe;

        // Nút kết thúc
        private Button btnKetThuc;

        public GiaoDienB2VN()
        {
            KhoiTaoGiaoDien();
        }

        private void KhoiTaoGiaoDien()
        {
            this.Text = "frmListbox";
            this.Size = new Size(520, 480);
            this.StartPosition = FormStartPosition.CenterScreen;
            this.FormBorderStyle = FormBorderStyle.FixedDialog;
            this.MaximizeBox = false;

            // Tiêu đề chữ hoa lớn ở giữa form
            lblTieuDe = new Label()
            {
                Text = "LISTBOX",
                Location = new Point(190, 10),
                AutoSize = true,
                Font = new Font("Arial", 16, FontStyle.Bold),
                ForeColor = Color.OrangeRed
            };

            // Bên trái: Nhập số và ListBox
            lblListBox = new Label() { Text = "Listbox", Location = new
Point(40, 45), AutoSize = true };

            txtNhap = new TextBox() { Location = new Point(40, 68), Size =
new Size(130, 23) };
            txtNhap.KeyPress += TxtNhap_KeyPress;
            txtNhap.KeyDown += TxtNhap_KeyDown;

            btnNhap = new Button() { Text = "Nhập", Location = new Point(40,
98), Size = new Size(130, 28) };
            btnNhap.Click += BtnNhap_Click;

            // ListBox hỗ trợ chọn nhiều phần tử (MultiSimple)
            lstSo = new ListBox()
```

```
            {
                Location = new Point(40, 135),
                Size = new Size(130, 240),
                SelectionMode = SelectionMode.MultiSimple,
                Font = new Font("Arial", 10, FontStyle.Regular)
            };

            // Bên phải: Các nút xử lý
            lblXuLy = new Label() { Text = "Xử lý Listbox", Location = new
Point(220, 45), AutoSize = true };

            int startX = 220;
            int startY = 68;
            int btnW = 230;
            int btnH = 28;
            int cachY = 6;

            btnTong = TaoNut("Tổng các phần tử trong List", startX, startY,
btnW, btnH);
            btnXoaDauCuoi = TaoNut("Xóa Phần tử đầu và cuối", startX, startY
+ (btnH + cachY) * 1, btnW, btnH);
            btnXoaDangChon = TaoNut("Xóa Phần tử đang chọn", startX, startY
+ (btnH + cachY) * 2, btnW, btnH);
            btnTang2 = TaoNut("Tăng mỗi phần tử lên 2", startX, startY +
(btnH + cachY) * 3, btnW, btnH);
            btnBinhPhuong = TaoNut("Thay bằng bình phương", startX, startY +
(btnH + cachY) * 4, btnW, btnH);
            btnChonChan = TaoNut("Chọn số chẵn", startX, startY + (btnH +
cachY) * 5, btnW, btnH);
            btnChonLe = TaoNut("Chọn số lẻ", startX, startY + (btnH + cachY)
* 6, btnW, btnH);

            // Gắn sự kiện các nút
            btnTong.Click += BtnTong_Click;
            btnXoaDauCuoi.Click += BtnXoaDauCuoi_Click;
            btnXoaDangChon.Click += BtnXoaDangChon_Click;
            btnTang2.Click += BtnTang2_Click;
            btnBinhPhuong.Click += BtnBinhPhuong_Click;
            btnChonChan.Click += BtnChonChan_Click;
            btnChonLe.Click += BtnChonLe_Click;

            // Nút KẾT THÚC
            btnKetThuc = new Button()
            {
                Text = "KẾT THÚC",
                Location = new Point(40, 390),
                Size = new Size(410, 35),
                Font = new Font("Arial", 10, FontStyle.Bold)
            };
            btnKetThuc.Click += (s, e) => this.Close();
```

```
            this.Controls.AddRange(new Control[] {
                lblTieuDe, lblListBox, txtNhap, btnNhap, lstSo,
                lblXuLy, btnTong, btnXoaDauCuoi, btnXoaDangChon, btnTang2,
                btnBinhPhuong, btnChonChan, btnChonLe, btnKetThuc
            });

            this.FormClosing += GiaoDienB2VN_FormClosing;
        }

        private Button TaoNut(string text, int x, int y, int w, int h)
        {
            return new Button()
            {
                Text = text,
                Location = new Point(x, y),
                Size = new Size(w, h),
                Font = new Font("Arial", 9, FontStyle.Regular)
            };
        }

        // Cập nhật lại toàn bộ danh sách hiển thị trên ListBox
        private void CapNhatListBox()
        {
            lstSo.BeginUpdate();
            lstSo.Items.Clear();
            foreach (int x in dsSo)
            {
                lstSo.Items.Add(x);
            }
            lstSo.EndUpdate();
        }

        // Chỉ cho phép nhập ký tự số nguyên
        private void TxtNhap_KeyPress(object sender, KeyPressEventArgs e)
        {
            if (!char.IsControl(e.KeyChar) && !char.IsDigit(e.KeyChar) &&
(e.KeyChar != '-'))
            {
                e.Handled = true;
            }
            if ((e.KeyChar == '-') && ((sender as TextBox).Text.IndexOf('-')
> -1))
            {
                e.Handled = true;
            }
        }

        // Nhấn phím Enter để thêm số nhanh
        private void TxtNhap_KeyDown(object sender, KeyEventArgs e)
        {
            if (e.KeyCode == Keys.Enter)
```

```
            {
                BtnNhap_Click(sender, e);
                e.SuppressKeyPress = true;
            }
        }

        // Nút Nhập: thêm số vào ListBox
        private void BtnNhap_Click(object sender, EventArgs e)
        {
            if (string.IsNullOrWhiteSpace(txtNhap.Text))
            {
                MessageBox.Show("Vui lòng nhập một số tự nhiên!", "Thông
báo");
                txtNhap.Focus();
                return;
            }

            if (int.TryParse(txtNhap.Text.Trim(), out int giaTri))
            {
                dsSo.Add(giaTri);
                CapNhatListBox();
                txtNhap.Clear();
                txtNhap.Focus();
            }
            else
            {
                MessageBox.Show("Giá trị nhập vào không hợp lệ!", "Lỗi");
                txtNhap.Focus();
            }
        }

        // Tính tổng các phần tử trong List, hiển thị lên MessageBox
        private void BtnTong_Click(object sender, EventArgs e)
        {
            if (dsSo.Count == 0)
            {
                MessageBox.Show("Danh sách đang trống!", "Thông báo");
                return;
            }

            long tong = XuLySoNguyen.TinhTong(dsSo);
            MessageBox.Show($"Tổng các phần tử trong List là: {tong}", "Kết
quả");
        }

        // Xóa phần tử đầu và cuối của listbox
        private void BtnXoaDauCuoi_Click(object sender, EventArgs e)
        {
            if (dsSo.Count == 0)
            {
                MessageBox.Show("Danh sách đang trống!", "Thông báo");
```

```
                return;
            }

            if (dsSo.Count == 1)
            {
                dsSo.RemoveAt(0);
            }
            else
            {
                // Xóa phần tử cuối trước rồi mới xóa phần tử đầu để không
bị lệch chỉ số
                dsSo.RemoveAt(dsSo.Count - 1);
                dsSo.RemoveAt(0);
            }

            CapNhatListBox();
        }

        // Xóa các phần tử đang chọn trong listbox
        private void BtnXoaDangChon_Click(object sender, EventArgs e)
        {
            if (lstSo.SelectedIndices.Count == 0)
            {
                MessageBox.Show("Vui lòng chọn ít nhất một phần tử để xóa!",
"Thông báo");
                return;
            }

            List<int> viTri = new List<int>();
            foreach (int idx in lstSo.SelectedIndices)
            {
                viTri.Add(idx);
            }
            viTri.Sort();

            // Duyệt ngược từ cuối về đầu để xóa an toàn
            for (int i = viTri.Count - 1; i >= 0; i--)
            {
                dsSo.RemoveAt(viTri[i]);
            }

            CapNhatListBox();
        }

        // Tăng giá trị mỗi phần tử lên 2
        private void BtnTang2_Click(object sender, EventArgs e)
        {
            if (dsSo.Count == 0)
            {
                MessageBox.Show("Danh sách đang trống!", "Thông báo");
                return;
```

```
        }

        XuLySoNguyen.TangLenHai(dsSo);
        CapNhatListBox();
    }

    // Thay mỗi giá trị bằng bình phương của chính nó
    private void BtnBinhPhuong_Click(object sender, EventArgs e)
    {
        if (dsSo.Count == 0)
        {
            MessageBox.Show("Danh sách đang trống!", "Thông báo");
            return;
        }

        XuLySoNguyen.ThayBangBinhPhuong(dsSo);
        CapNhatListBox();
    }

    // Thực hiện chọn các phần tử trong listbox là số chẵn
    private void BtnChonChan_Click(object sender, EventArgs e)
    {
        if (dsSo.Count == 0)
        {
            MessageBox.Show("Danh sách đang trống!", "Thông báo");
            return;
        }

        lstSo.ClearSelected();
        for (int i = 0; i < dsSo.Count; i++)
        {
            if (XuLySoNguyen.LaSoChan(dsSo[i]))
            {
                lstSo.SetSelected(i, true);
            }
        }
    }

    // Thực hiện chọn các phần tử trong listbox là số lẻ
    private void BtnChonLe_Click(object sender, EventArgs e)
    {
        if (dsSo.Count == 0)
        {
            MessageBox.Show("Danh sách đang trống!", "Thông báo");
            return;
        }

        lstSo.ClearSelected();
        for (int i = 0; i < dsSo.Count; i++)
        {
            if (XuLySoNguyen.LaSoLe(dsSo[i]))
```

```
                {
                    lstSo.SetSelected(i, true);
                }
            }
        }

        // Thoát có xác nhận từ người dùng
        private void GiaoDienB2VN_FormClosing(object sender,
FormClosingEventArgs e)
        {
            DialogResult hoi = MessageBox.Show("Bạn có muốn kết thúc chương
trình không?", "Xác nhận", MessageBoxButtons.YesNo,
MessageBoxIcon.Question);
            if (hoi == DialogResult.No)
            {
                e.Cancel = true;
            }
        }
    }
}
```

Program.cs

```
using System;
using System.Windows.Forms;

namespace BTVN2
{
    static class Program
    {
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new GiaoDienB2VN());
        }
    }
}
```

3. Giải thích bài làm

Lớp XuLySoNguyen chứa các hàm xử lý tính toán số học độc lập (TinhTong(), TangLenHai(), ThayBangBinhPhuong(), LaSoChan(), LaSoLe()). Việc tách riêng này giúp giữ cho Form giao diện ngắn gọn và phân định rõ trách nhiệm xử lý logic.

Quản lý danh sách bằng List<int> dsSo: Dữ liệu số nguyên được lưu giữ trong danh sách dsSo. Ban đầu danh sách để trống để người dùng tự nhập. Khi có thao tác thêm, xóa, sửa, hàm CapNhatListBox() sẽ đồng bộ dữ liệu vào lstSo.Items với các lệnh BeginUpdate() và EndUpdate().

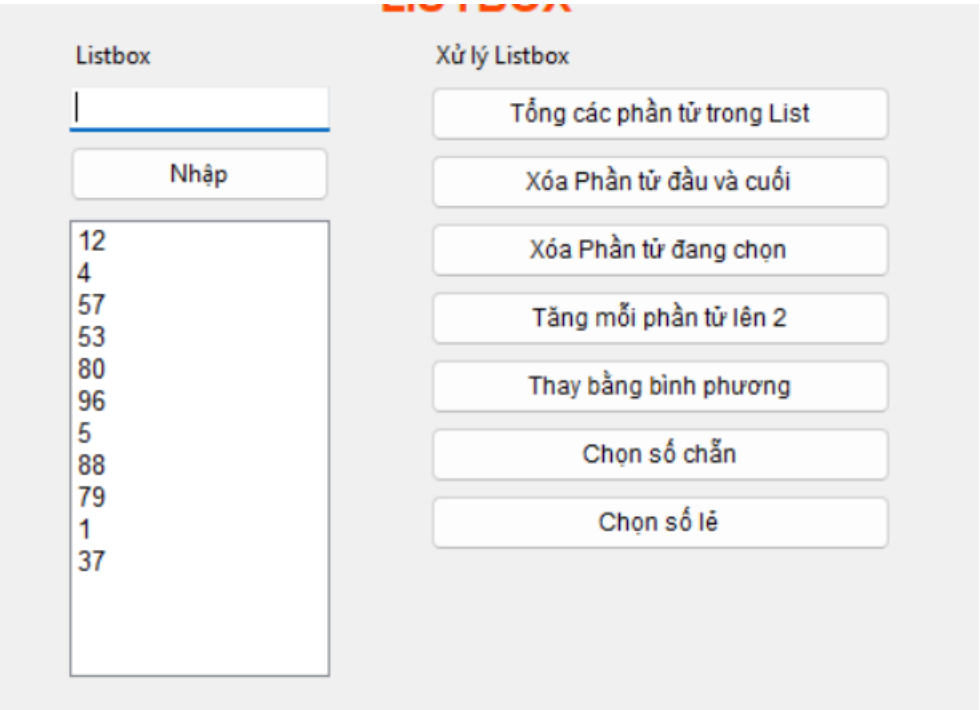
Kỹ thuật xóa an toàn:

- Với nút **Xóa Phần tử đầu và cuối**: Xóa phần tử cuối ở vị trí Count - 1 trước, sau đó mới xóa phần tử ở vị trí 0 để tránh bị lệch chỉ số mảng.
- Với nút **Xóa Phần tử đang chọn**: Sao chép các chỉ số được chọn ra mảng riêng, sắp xếp và duyệt vòng lặp for ngược từ cuối về đầu.

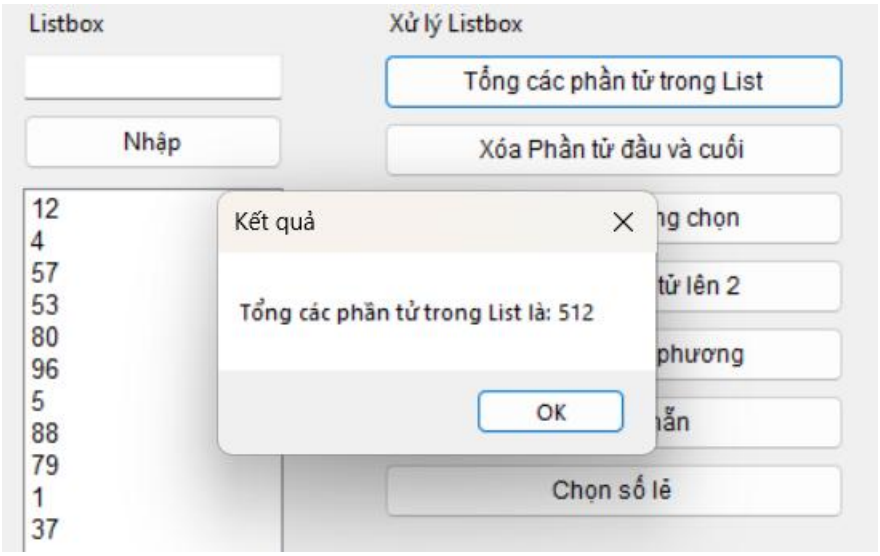
Thao tác chọn tự động qua SetSelected(): Trước khi chọn, chương trình gọi lstSo.ClearSelected() để xóa trạng thái chọn cũ; sau đó kiểm tra từng số bằng hàm của lớp nghiệp vụ, nếu thỏa mãn sẽ kích hoạt chọn bằng lstSo.SetSelected(i, true)

4. Kết quả

Nhập dữ liệu vào ListBox: Nhập lần lượt các số → Nhấn Enter hoặc nút nhập → Các số xuất hiện

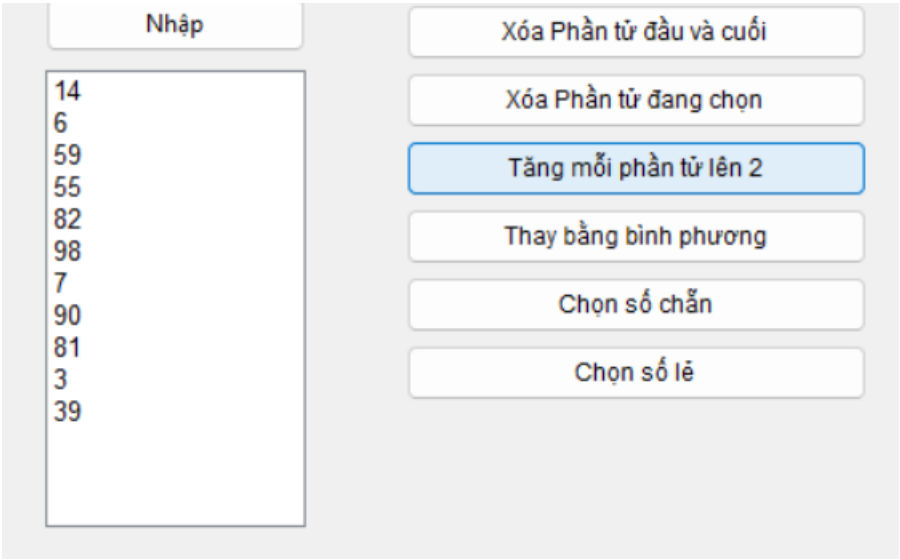


Tính tổng các phần tử: Bấm nút **Tổng các phần tử trong List** → Hiển thị chính xác tổng các số vừa nhập

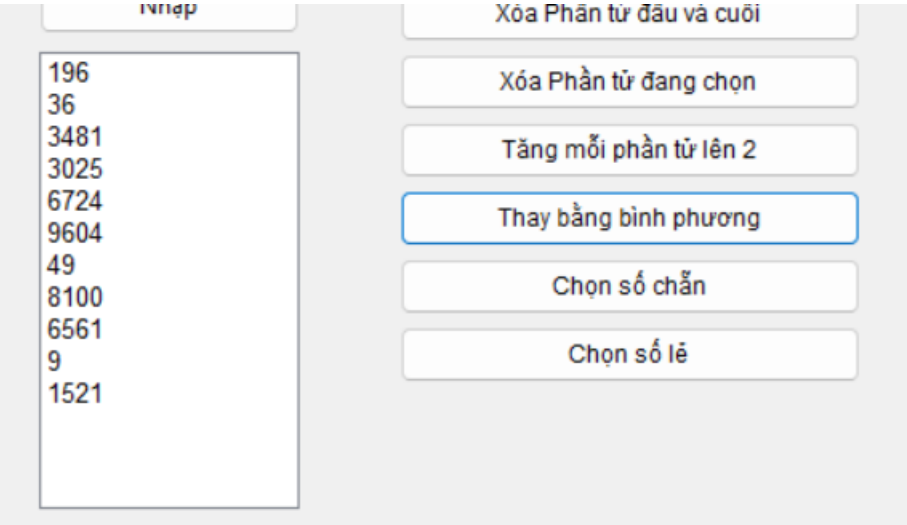


Tăng giá trị và bình phương:

Bấm **Tăng mỗi phần tử lên 2** → Toàn bộ số trong ListBox đều tang them 2 đơn vị

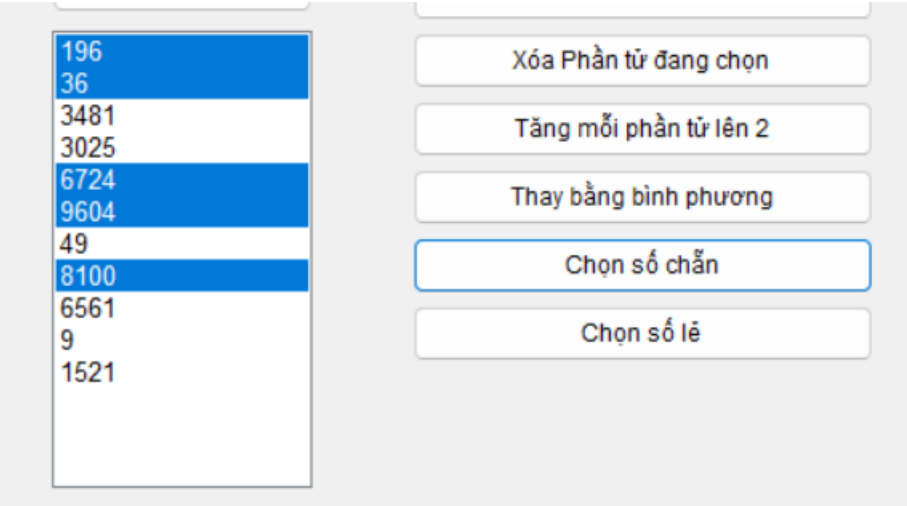


Bấm **Thay bằng bình phương** → Mỗi số được gán lại bằng giá trị bình phương của chính nó

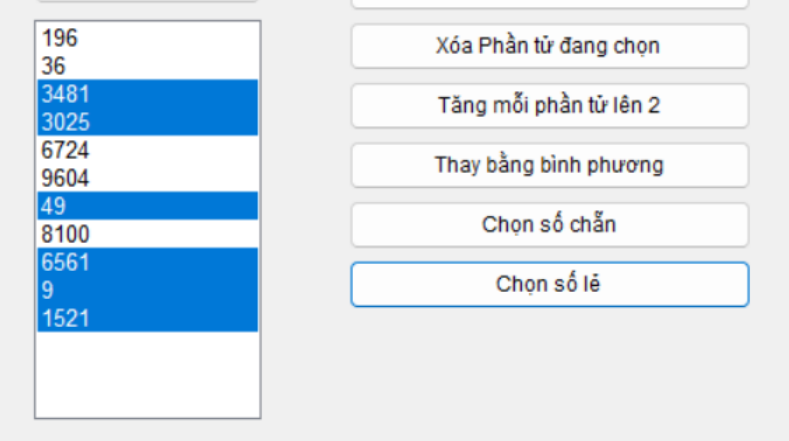


Chọn số chẵn và số lẻ:

Bấm **Chọn số chẵn** → Các phần tử chia hết cho 2 được tự động bôi xanh

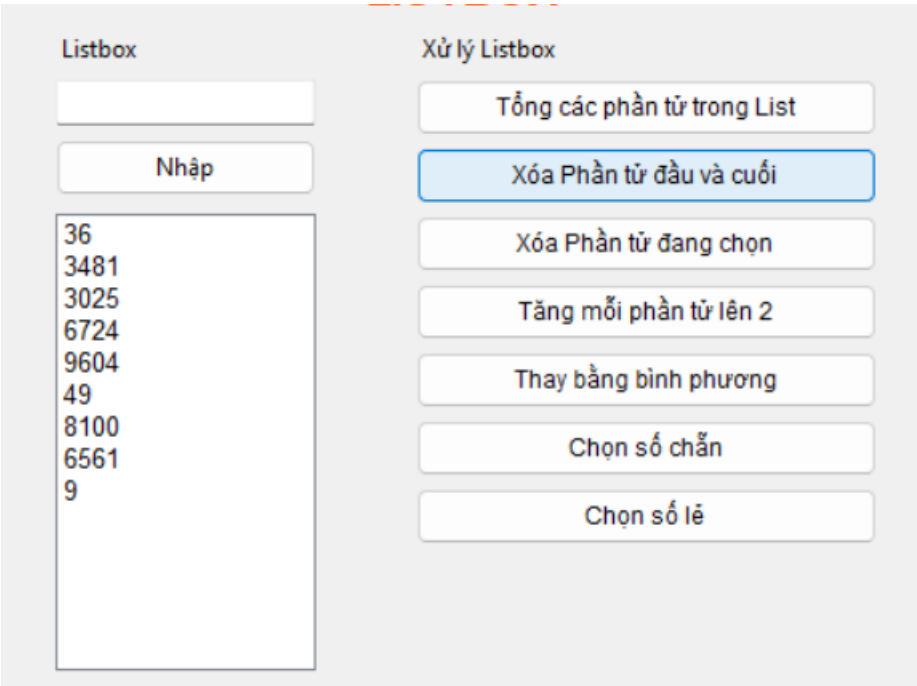


Bấm **Chọn số lẻ** → Toàn bộ các phần tử lẻ được bôi xanh

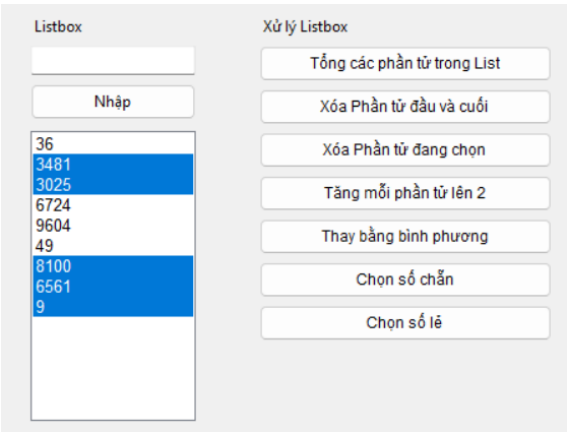


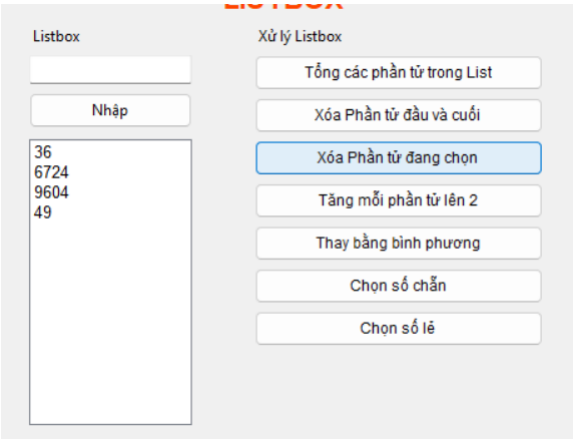
Xóa đầu cuối và xóa đang chọn:

Bấm **Xóa phần tử đầu và cuối** → Số đầu tiên và số cuối cùng trên danh sách bị gỡ bỏ



Nhập số bất kỳ → Bấm **Xóa phần tử đang chọn** → Các phần tử được chọn bị xóa





Bài 3:

1. **Yêu cầu đề**

− Thiết kế giao diện gồm TreeView bên trái và các ô nhập First Name, Last Name, nút Add, Exit bên phải.

− Khi form được mở lên (Load), TreeView tự động khởi tạo chứa tất cả các nút ký tự từ A → Z.

− Khi thêm một người mới vào danh bạ (bấm nút Add), chương trình tự động đưa người này vào TreeView tại vị trí nút con tương ứng với chữ cái đầu của First Name theo định dạng: FirstName, LastName.

− Nút Exit: Thoát khỏi chương trình có hộp thoại xác nhận từ người dùng.

2. **Mã nguồn**

Bai3VN.cs

```
using System;

namespace BTVN3
{
    public class NguoiDanhBa
    {
        public string FirstName { get; set; }
        public string LastName { get; set; }

        public NguoiDanhBa()
        {
            FirstName = "";
            LastName = "";
        }

        public NguoiDanhBa(string firstName, string lastName)
        {
            FirstName = firstName;
            LastName = lastName;
        }

        // Định dạng hiển thị trên nút con TreeView: "FirstName, LastName"
        public string LayChuoiHienThi()
        {
            return FirstName + ", " + LastName;
        }

        // Lấy chữ cái đầu tiên của First Name để phân nhóm A đến Z
        public char LayKyTuDau()
        {
```

```
            if (string.IsNullOrWhiteSpace(FirstName)) return '#';

            // Chuyển ký tự đầu tiên về dạng chữ hoa không dấu tiếng Anh cơ bản
            char c = FirstName.Trim().ToUpper()[0];

            // Xử lý các chữ cái tiếng Việt có dấu về chữ cái gốc tương ứng
            switch (c)
            {
                case 'Á': case 'À': case 'Ả': case 'Ã': case 'Ạ':
                case 'Ă': case 'Ắ': case 'Ằ': case 'Ẳ': case 'Ẵ': case 'Ặ':
                case 'Â': case 'Ấ': case 'Ầ': case 'Ẩ': case 'Ẫ': case 'Ậ':
                    return 'A';
                case 'Đ':
                    return 'D';
                case 'É': case 'È': case 'Ẻ': case 'Ẽ': case 'Ẹ':
                case 'Ê': case 'Ế': case 'Ề': case 'Ể': case 'Ễ': case 'Ệ':
                    return 'E';
                case 'Í': case 'Ì': case 'Ỉ': case 'Ĩ': case 'Ị':
                    return 'I';
                case 'Ó': case 'Ò': case 'Ỏ': case 'Õ': case 'Ọ':
                case 'Ô': case 'Ố': case 'Ồ': case 'Ổ': case 'Ỗ': case 'Ộ':
                case 'Ơ': case 'Ớ': case 'Ờ': case 'Ở': case 'Ỡ': case 'Ợ':
                    return 'O';
                case 'Ú': case 'Ù': case 'Ủ': case 'Ũ': case 'Ụ':
                case 'Ư': case 'Ứ': case 'Ừ': case 'Ử': case 'Ữ': case 'Ự':
                    return 'U';
                case 'Ý': case 'Ỳ': case 'Ỷ': case 'Ỹ': case 'Ỵ':
                    return 'Y';
                default:
                    return c;
            }
        }
    }
}
```

GiaoDienB3VN.cs

```
using System;
using System.Drawing;
using System.Windows.Forms;

namespace BTVN3
{
    public class GiaoDienB3VN : Form
    {
        private TreeView trvDanhBa;
        private Label lblFirstName, lblLastName;
        private TextBox txtFirstName, txtLastName;
        private Button btnAdd, btnExit;
```

```
        public GiaoDienB3VN()
        {
            KhoiTaoGiaoDien();
        }

        private void KhoiTaoGiaoDien()
        {
            this.Text = "frmBTVN2";
            this.Size = new Size(540, 480);
            this.StartPosition = FormStartPosition.CenterScreen;
            this.FormBorderStyle = FormBorderStyle.FixedDialog;
            this.MaximizeBox = false;

            // Cây danh bạ bên trái
            trvDanhBa = new TreeView()
            {
                Location = new Point(15, 15),
                Size = new Size(200, 410),
                Font = new Font("Arial", 11, FontStyle.Regular)
            };

            // Khu vực nhập liệu bên phải
            lblFirstName = new Label()
            {
                Text = "First Name",
                Location = new Point(240, 45),
                AutoSize = true,
                Font = new Font("Arial", 10, FontStyle.Regular)
            };

            txtFirstName = new TextBox()
            {
                Location = new Point(330, 42),
                Size = new Size(170, 23),
                Font = new Font("Arial", 10, FontStyle.Regular)
            };

            lblLastName = new Label()
            {
                Text = "Last Name",
                Location = new Point(240, 95),
                AutoSize = true,
                Font = new Font("Arial", 10, FontStyle.Regular)
            };

            txtLastName = new TextBox()
            {
                Location = new Point(330, 92),
                Size = new Size(170, 23),
                Font = new Font("Arial", 10, FontStyle.Regular)
```

```
            };

            btnAdd = new Button()
            {
                Text = "Add",
                Location = new Point(410, 140),
                Size = new Size(90, 32),
                Font = new Font("Arial", 10, FontStyle.Regular)
            };
            btnAdd.Click += BtnAdd_Click;

            btnExit = new Button()
            {
                Text = "Exit",
                Location = new Point(410, 210),
                Size = new Size(90, 32),
                Font = new Font("Arial", 10, FontStyle.Regular)
            };
            btnExit.Click += (s, e) => this.Close();

            this.Controls.AddRange(new Control[] {
                trvDanhBa, lblFirstName, txtFirstName,
                lblLastName, txtLastName, btnAdd, btnExit
            });

            // Gán sự kiện Load và Closing
            this.Load += GiaoDienB3VN_Load;
            this.FormClosing += GiaoDienB3VN_FormClosing;
        }

        // Khi mở Form: khởi tạo các nút từ A đến Z trên TreeView
        private void GiaoDienB3VN_Load(object sender, EventArgs e)
        {
            trvDanhBa.Nodes.Clear();
            for (char c = 'A'; c <= 'Z'; c++)
            {
                TreeNode node = new TreeNode(c.ToString());
                node.Name = c.ToString(); // Đặt name để dễ tìm kiếm nút
                trvDanhBa.Nodes.Add(node);
            }
        }

        // Nút Add: thêm người mới vào đúng nút chữ cái đầu của First Name
        private void BtnAdd_Click(object sender, EventArgs e)
        {
            string firstName = txtFirstName.Text.Trim();
            string lastName = txtLastName.Text.Trim();

            // Kiểm tra không để trống ô nhập
            if (string.IsNullOrEmpty(firstName) ||
string.IsNullOrEmpty(lastName))
```

```
            {
                MessageBox.Show("Vui lòng nhập đầy đủ First Name và Last
Name!", "Thông báo", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                txtFirstName.Focus();
                return;
            }

            NguoiDanhBa nguoiMoi = new NguoiDanhBa(firstName, lastName);
            char kyTuDau = nguoiMoi.LayKyTuDau();

            // Tìm nút cha tương ứng với ký tự đầu trên TreeView
            TreeNode[] danhSachNodeTimThay =
trvDanhBa.Nodes.Find(kyTuDau.ToString(), false);

            if (danhSachNodeTimThay.Length > 0)
            {
                TreeNode nodeCha = danhSachNodeTimThay[0];

                // Kiểm tra xem người này đã tồn tại trong nhánh chữ cái này
chưa
                string chuoiHienThi = nguoiMoi.LayChuoiHienThi();
                foreach (TreeNode nodeCon in nodeCha.Nodes)
                {
                    if (string.Compare(nodeCon.Text, chuoiHienThi, true) ==
0)
                    {
                        MessageBox.Show("Người này đã có trong danh bạ!",
"Thông báo", MessageBoxButtons.OK, MessageBoxIcon.Information);
                        return;
                    }
                }

                // Thêm nút con mới vào nút cha
                TreeNode nodeConMoi = new TreeNode(chuoiHienThi);
                nodeCha.Nodes.Add(nodeConMoi);

                // Mở rộng nút cha để người dùng thấy ngay kết quả
                nodeCha.Expand();

                // Chọn nút vừa thêm
                trvDanhBa.SelectedNode = nodeConMoi;

                // Xóa trắng ô nhập và đặt con trỏ chuột quay lại First Name
                txtFirstName.Clear();
                txtLastName.Clear();
                txtFirstName.Focus();
            }
            else
            {
                MessageBox.Show("Tên phải bắt đầu bằng chữ cái từ A đến Z!",
"Lỗi ký tự", MessageBoxButtons.OK, MessageBoxIcon.Warning);
```

```
                txtFirstName.Focus();
            }
        }

        // Xác nhận trước khi đóng chương trình
        private void GiaoDienB3VN_FormClosing(object sender,
FormClosingEventArgs e)
        {
            DialogResult hoi = MessageBox.Show("Bạn có muốn thoát chương
trình danh bạ không?", "Xác nhận", MessageBoxButtons.YesNo,
MessageBoxIcon.Question);
            if (hoi == DialogResult.No)
            {
                e.Cancel = true;
            }
        }
    }
}
```

Program.cs

```
using System;
using System.Windows.Forms;

namespace BTVN3
{
    static class Program
    {
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new GiaoDienB3VN());
        }
    }
}
```

3. Giải thích bài làm

Lớp NguoiDanhBa lưu FirstName và LastName. Trong đó, hàm LayChuoiHienThi() ghép thành chuỗi "FirstName, LastName", còn hàm LayKyTuDau() lấy chữ cái đầu tiên của First Name và đổi các ký tự tiếng Việt có dấu về chữ cái chuẩn A → Z để gom nhóm.

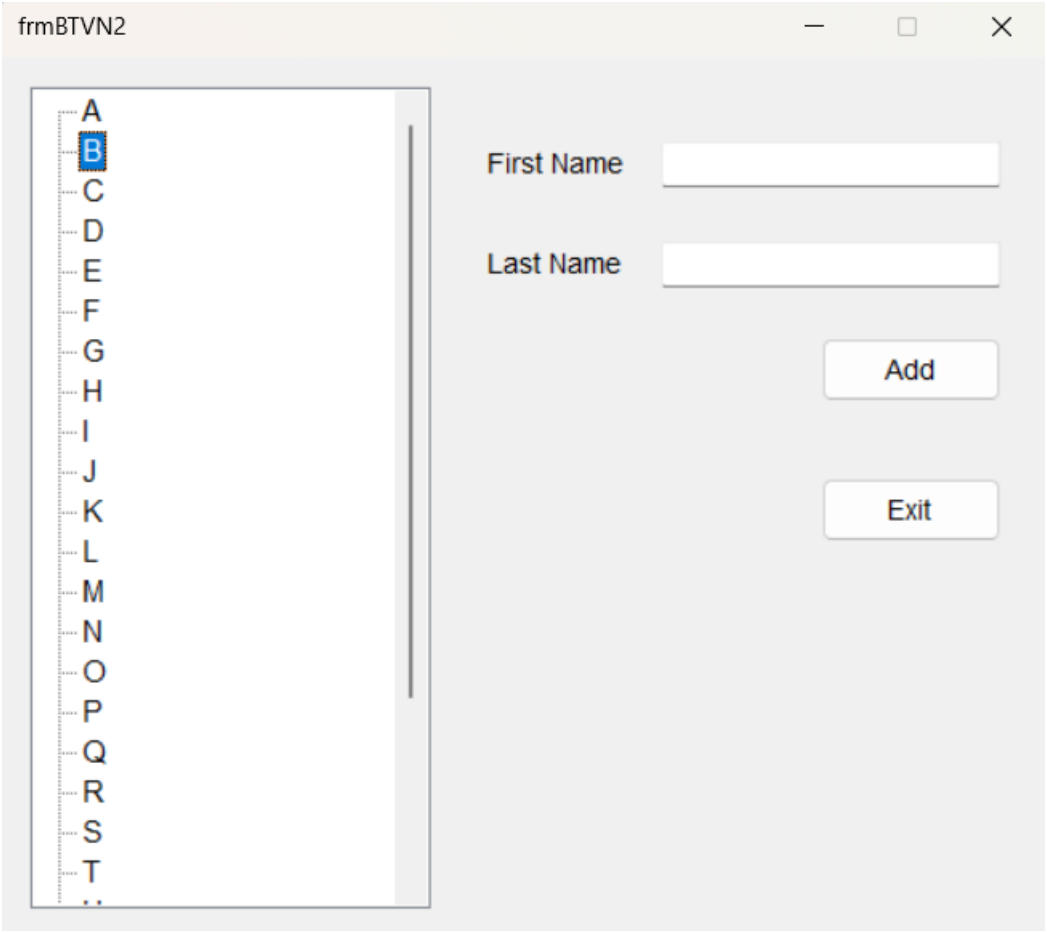
Tạo bảng chữ cái khi mở Form: Sự kiện Load dùng vòng lặp ký tự for (char c = 'A'; c <= 'Z'; c++) tạo sẵn 26 nút gốc từ A đến Z trên TreeView.

Thêm người vào danh bạ (Nút Add): Sau khi kiểm tra người dùng không để trống ô nhập, chương trình lấy chữ cái đầu tiên của First Name rồi dùng lệnh Nodes.Find() tìm nhanh đến nút cha tương ứng. Tiếp theo, chương trình tạo một nút con mới gắn vào nhánh đó, đồng thời gọi Expand() để bung nhánh ra giúp người dùng nhìn thấy ngay kết quả.

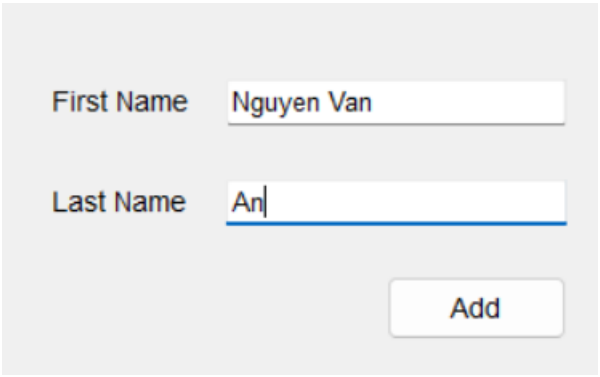
Nút gọi lệnh đóng form, sự kiện FormClosing sẽ hiện hộp thoại hỏi xác nhận yes/no trước khi thoát hẳn.

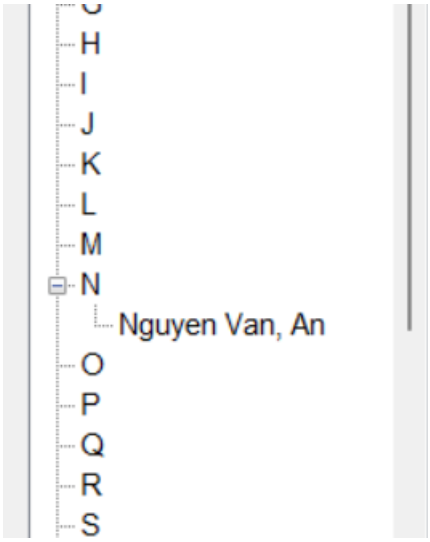
4. Kết quả

Khởi động Form:



Thêm người vào danh bạ: Nhập FirstName, LastName → Nhấn nút Add → Nhánh trùng chữ đầu với LastName tự động mở rộng và xuất hiện nút con

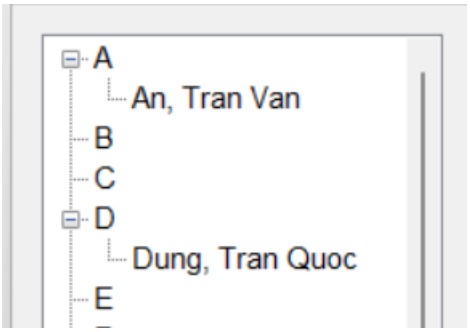




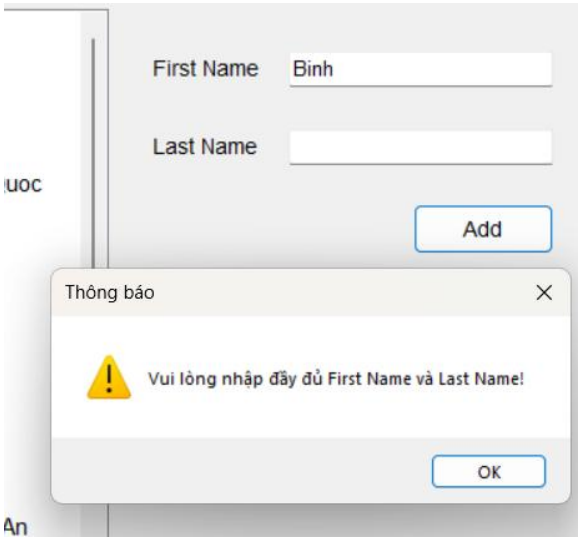
Thêm nhiều người thuộc các nhóm khác nhau:

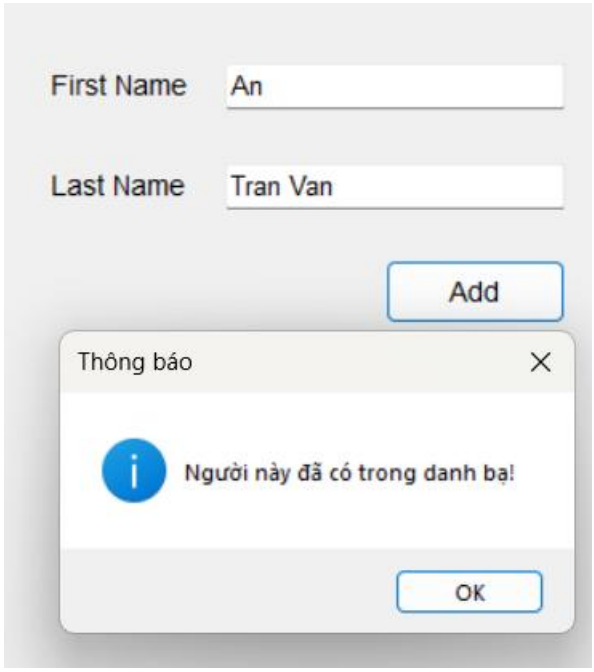
Nhập FirstName: An, LastName: Tran Van → Nút nhánh A mở ra chứa An, Nguyen Van

Nhập FirtNameL Dung, LastNAme: Tran Quoc → Nut nhánh D mở ra chứa Dung, Tran Quoc



Kiểm tra dữ liệu rỗng và trùng: Để trống 1 trong 2 ô rồi bấm Add → Xuất hiện thông báo lỗi. Nếu nhập lại tên đã nhập → Xuất hiện thông báo lỗi


First Name
Binh
Last Name
Add
Thông báo
Vui lòng nhập đầy đủ First Name và Last Name!
OK


First Name
An
Last Name
Tran Van
Add
Thông báo
Người này đã có trong danh bạ!
OK